

FROGALONE

José Luis García García

I.E.S. Kursaal

Proyecto integrado 2025



Índice

Introducción y presentación del proyecto	- 3 -
Requisitos hardware y software del proyecto	- 4 -
Análisis / Diseño de la aplicación	- 4 -
DIAGRAMAS Y DISEÑO	- 7 -
Manual técnico	- 11 -
ESTRUCTURA DEL PROYECTO	- 11 -
FUNCIONES PRINCIPALES.....	- 14 -
FUNCIONES DE NIVELES	- 22 -
CLASES DE OBJETOS.....	- 33 -
SCRIPT DE LA PÁGINA	- 43 -
PÁGINA ALTERNA:.....	- 52 -
Manual del usuario	- 55 -
COMO INICIAR EL JUEGO	- 55 -
MENÚS E INTERFAZ	- 55 -
Propuesta de ampliación del proyecto	- 61 -
Estudio de viabilidad del proyecto	- 62 -
Conclusión.....	- 63 -

Introducción y presentación del proyecto

Este Proyecto consta de la creación de un videojuego de plataformas en 2D desarrollado con el framework Phaser y el lenguaje TypeScript.

Este proyecto tiene como objetivo principal diseñar, implementar y documentar un videojuego interactivo que recuerde a los elementos clásicos del genero de plataformas, también adaptado a la jugabilidad a través de cualquier dispositivo, pudiendo ser jugado desde web a través de un ordenador y un dispositivo móvil.

El videojuego está orientado a usuarios casuales y servirá como una muestra funcional de las capacidades de desarrollo frontend y backend en un entorno de programación web ya que, no es necesaria la descarga de ningún archivo para poder jugar al videojuego. A través de este proyecto, se pretende demostrar el dominio de herramientas de desarrollo modernas como la planificación estructurada de un sistema interactivo, la implementación de buenas prácticas en diseño, usabilidad, accesibilidad, rendimiento... junto con uno de los géneros más antiguos de los videojuegos lo que da una experiencia enriquecedora a nuevos usuarios en esta clase de títulos, así como a los usuarios más experimentados.

Entre los objetivos específicos del proyecto se encuentran:

- Diseñar una arquitectura modular y escalable para el videojuego.
- Implementar una jugabilidad intuitiva y accesible con niveles progresivos.
- Utilizar el framework de Phaser como motor de juego para gestionar gráficos, físicas y lógica del juego.
- Integrar TypeScript para garantizar un desarrollo moderno, tipado, mantenible y robusto.
- Desplegar el videojuego a formato público donde cualquier persona pueda ser capaz de probarlo.
- Documentar exhaustivamente tanto el proceso de diseño como el desarrollo así igual como el producto final.

Requisitos hardware y software del proyecto

El desarrollo y ejecución del videojuego “FrogAlone” se ha llevado a cabo utilizando tecnologías modernas que permiten una experiencia fluida tanto en entornos de desarrollo como en producción. A continuación, se detallan los requisitos mínimos y recomendados para el correcto funcionamiento del proyecto.

SISTEMA OPERATIVO

El videojuego está diseñado para ejecutarse en navegadores web modernos, por lo que es compatible con múltiples sistemas operativos. Sin embargo, para el entorno de desarrollo se ha utilizado:

- Sistema operativo principal: Windows 11 Pro (versión 23H2)
- Alternativas compatibles:
- macOS Ventura 13.5 o superior
 - Ubuntu 22.04 LTS o superior

SOFTWARE UTILIZADO PARA EL DESARROLLO DEL VIDEOJUEGO

- Motor de juego: Phaser 3 (versión 3.60 o superior)
- Lenguaje de programación: TypeScript (versión 5.2 o superior)
- Entorno de desarrollo: Visual Studio Code
- Gestor de paquetes: Node.js (versión 20.x) con npm
- Servidor local para pruebas: Live Server
- Control de versiones: Git y GitHub

NAVEGADORES COMPATIBLES

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Análisis / Diseño de la aplicación

ANÁLISIS DE REQUISITOS

Requisitos funcionales:

- El jugador podrá controlar un personaje principal que se desplaza lateralmente y salta entre plataformas.
- El juego incluirá enemigos, obstáculos y objetos coleccionables.

- Se implementarán diferentes niveles con dificultad progresiva.
- El juego mostrará una interfaz con información como puntuación, vidas y tiempo restante.
- El progreso del jugador se guardará localmente mediante el almacenamiento interno del navegador (localStorage).

Requisitos no funcionales:

- El juego debe ejecutarse en navegadores modernos sin necesidad de instalación alguna.
- El rendimiento debe ser bueno y mantenerse estable en equipos con hardware básico.
- El código debe estar modularizado y documentado para facilitar su mantenimiento.

DISEÑO DEL MODELO DE DATOS

Aunque el videojuego no requiere de una base de datos compleja, se ha diseñado un modelo de datos para gestionar la información de las partidas en el almacenamiento interno del navegador o localStorage siendo este un archivo JSON que se añade al iniciar el videojuego.

El archivo estaría compuesto por la siguiente estructura de datos:

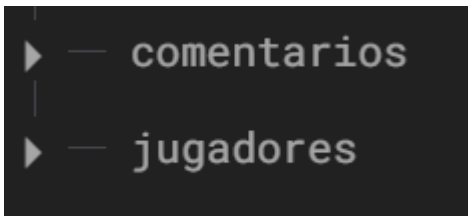
```
{
  "music": (valor booleano),
  "effects": (valor booleano),
  "levels": {
    "level01": {
      "score": (valor numérico),
      "isPassed": (valor booleano),
    },
    ...
  }
}
```

Siendo este el JSON que se genera al iniciar una partida, vemos que es una estructura de datos muy simple donde tenemos:

- “music”: variable que nos indica las preferencias del usuario en cuanto a reproducir, o no, música.
- “effects”: variable que nos indica las preferencias del usuario en cuanto a reproducir, o no, música.
- “levels”: en este objeto se encuentran todos los niveles existentes en el videojuego.
- “levelXX”: en este objeto se encuentran los valores para cada nivel del videojuego.
- “score”: puntuación numérica calculada al finalizar si el jugador pasa el nivel.
- “isPassed”: valor que nos indica si el jugador ha pasado, o no, el nivel al que pertenece.

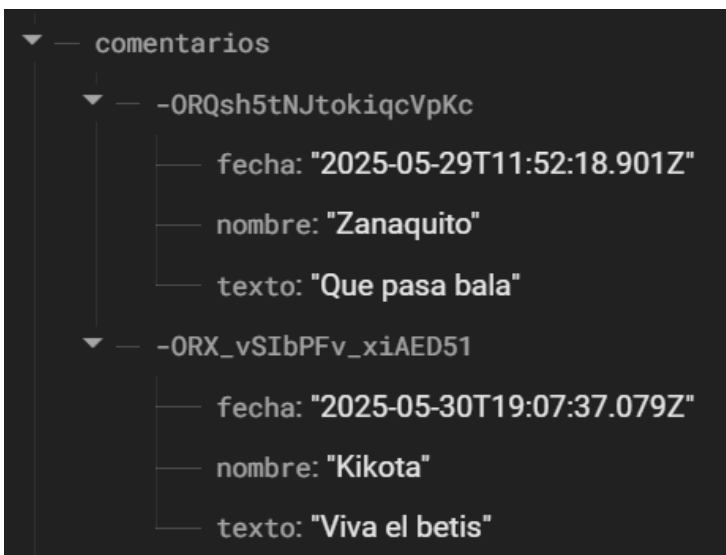
En el modelo de datos en la nube tenemos una base de datos no relacional utilizada a través de Firebase para un despliegue global.

En Firebase hemos elegido en el apartado consola un “Realtime Database” para que los datos sean sincronizados lo antes posible para que el usuario pueda visualizarlos de manera satisfactoria. Tenemos el siguiente contenido dentro de la base de datos:



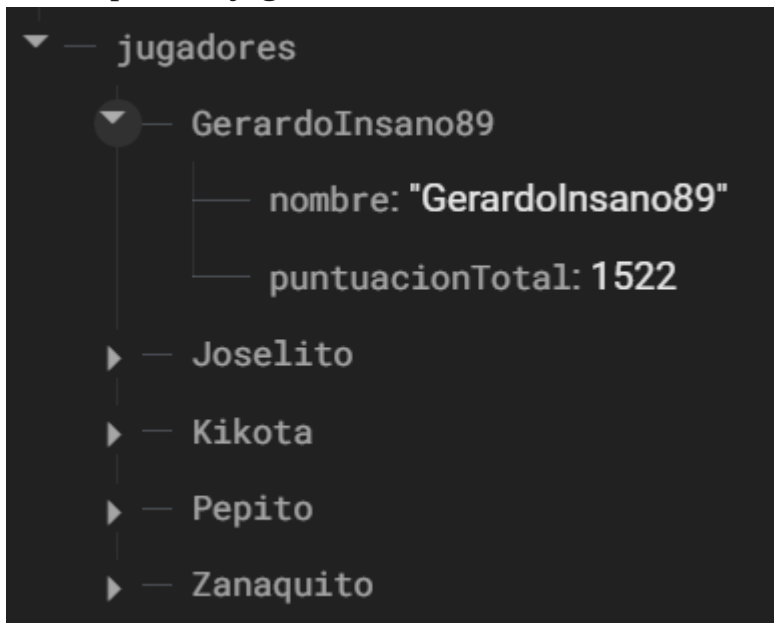
Donde cada una de las opciones tiene características diferentes.

Veamos la tabla comentarios:



En esta tabla encontramos un id autogenerado para cada comentario de la aplicación, dentro del id vemos la fecha a la que se ha subido el comentario, el nombre del autor y el texto que escribió, así para cada comentario.

En el apartado jugadores vemos:



En este apartado tenemos el id del jugador que ha subido su puntuación y dentro de él, el nombre de usuario y su puntuación total de la partida. Los jugadores que mayor puntuación tengan serán recogidos para mostrar su nombre y puntos en un ranking global más tarde.

DIAGRAMAS Y DISEÑO

Para una evolución de la aplicación correcta se han decidido hacer diagramas que han ido acompañando el desarrollo hasta la actualidad y se quieren preservar con fines de que se siga siguiendo la lógica de estos para el futuro.

Con esto se verán los diagramas de casos de uso, diseño de niveles en formato boceto y planteamiento de la base de datos futura dentro de un servidor.

Diseño de niveles:

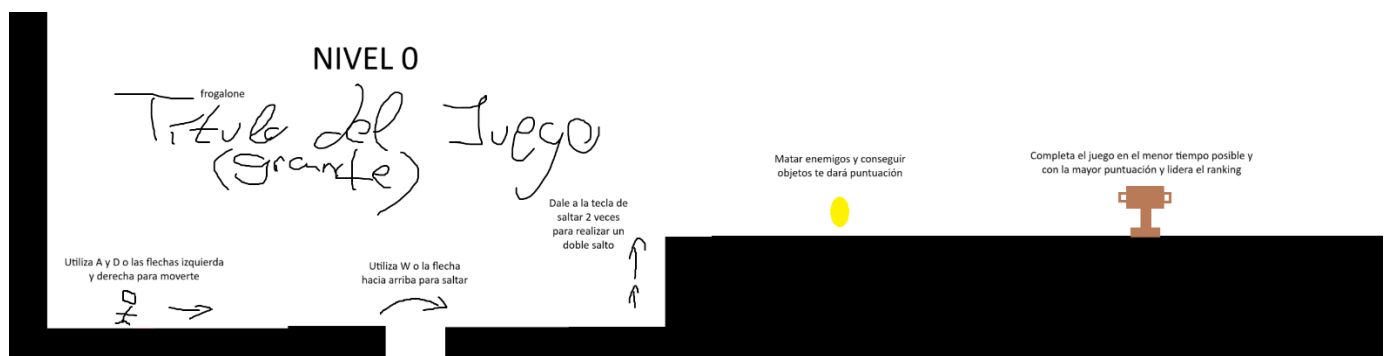
Veremos primero los bocetos de diseño de niveles:

Estos bocetos deben ser garabatos, donde se tiene que explicar la dirección del jugador con flechas. Si hay textos hay que escribirlos. Los enemigos deben ser un

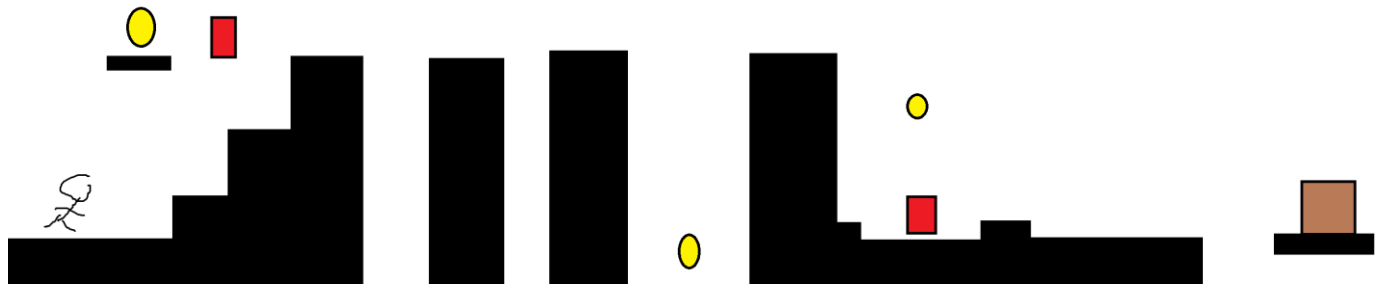
cuadrado rojo, las frutas deben ser un óvalo amarillo y la copa puede ser un cuadrado marrón.

Estos bocetos están sujetos a cambio dentro del desarrollo y puede no ser la fase final del nivel, en todo caso será la base para la construcción de este.

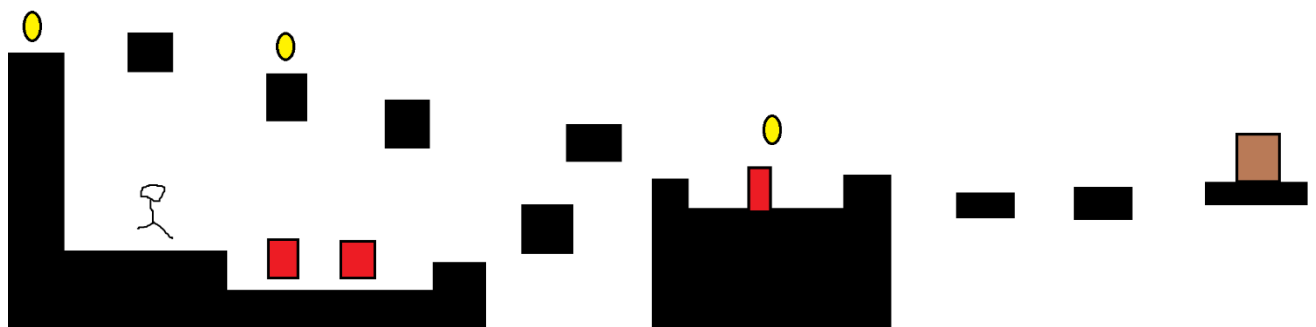
Nivel 1:



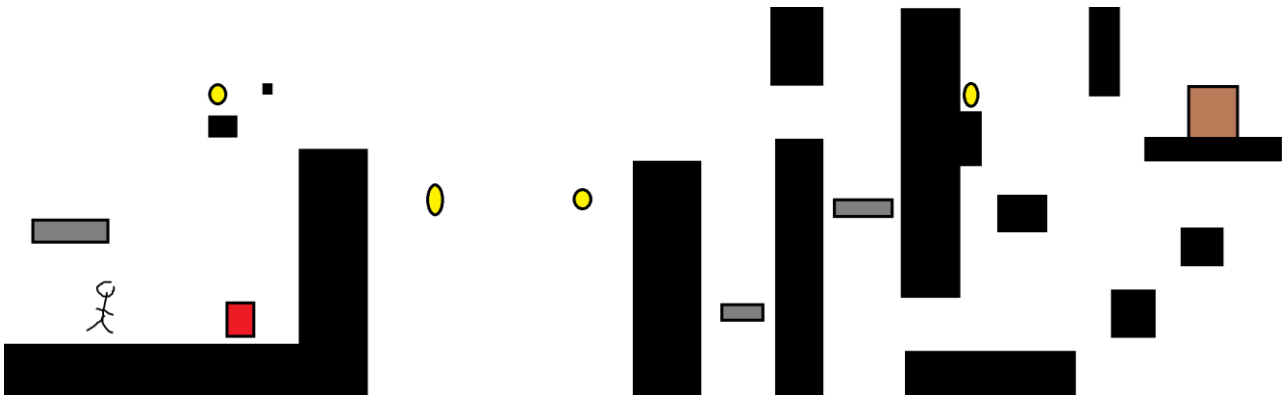
Nivel 2:



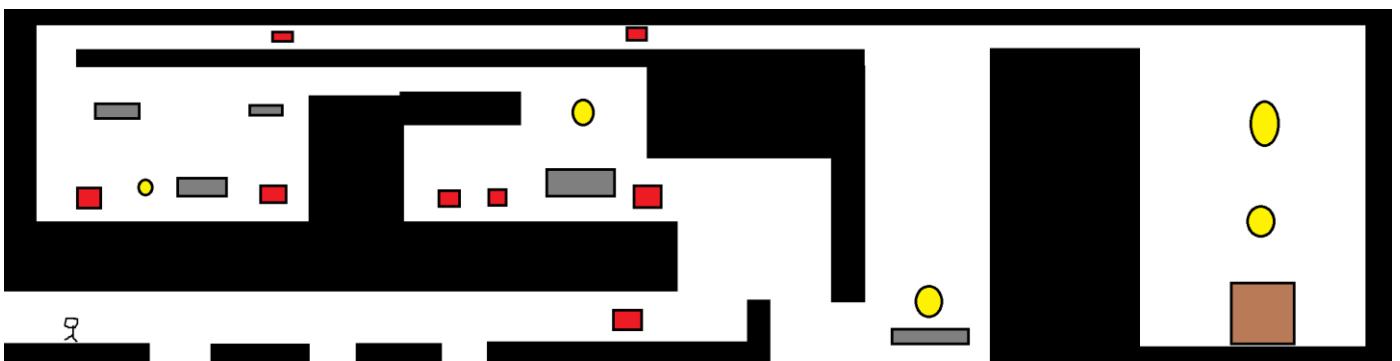
Nivel 3:



Nivel 4:

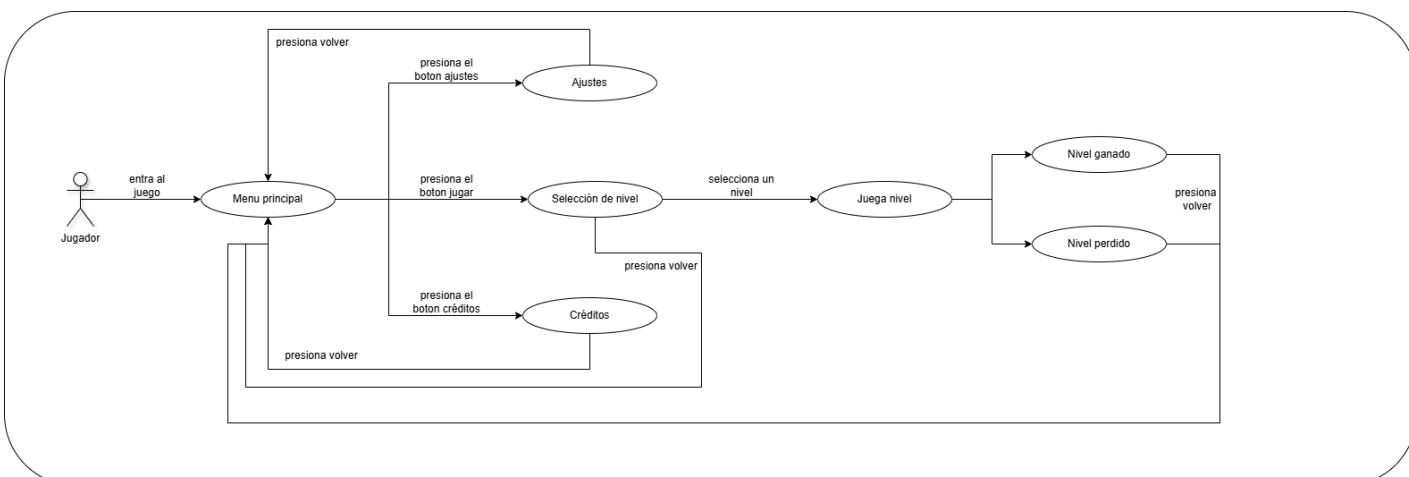


Nivel 5:



A continuación, vamos a ver el diagrama de casos de uso, a través de este diagrama tendremos claro la continuación que tendrá el usuario que la utilice a través de la aplicación, nos dirá lo que puede o no hacer en todo momento.

Diagrama de casos de uso:



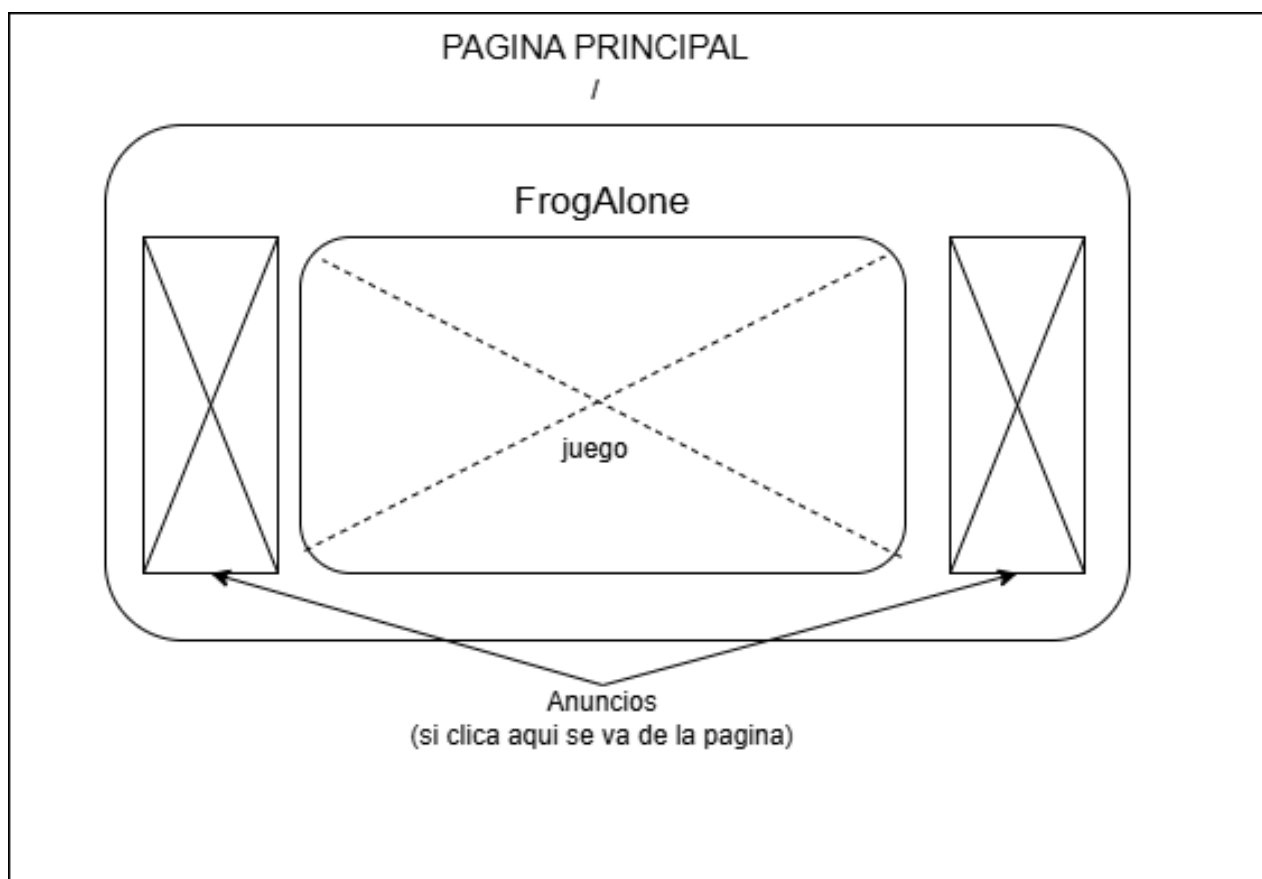
En este diagrama vemos que el flujo de usuario se centra, principalmente, en los botones de retorno al menú principal y en jugar un nivel.

Al haber únicamente una sola entidad de usuario no es necesario hacer otros casos de uso ya que, en este momento, no hay funciones de registro o inicio de sesión.

El diseño fue realizado en la aplicación draw.io y puede ser cambiado fácilmente, el archivo se encuentra en la carpeta “DisenoNiveles”.

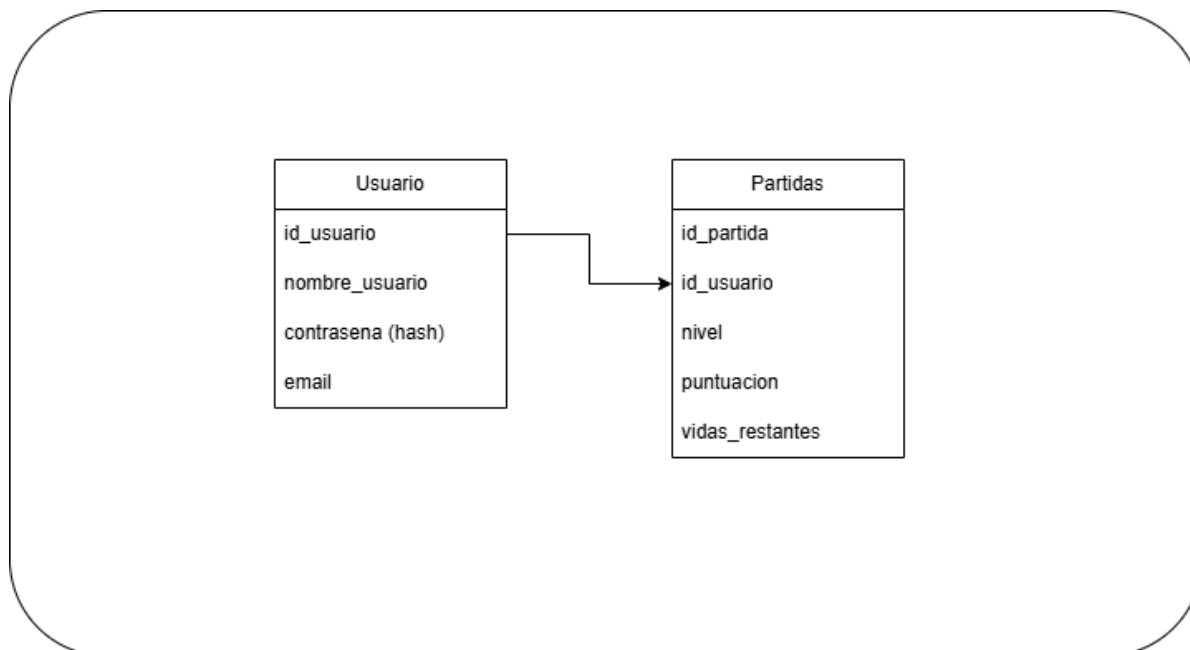
Diseño del sitio web:

Para el diseño del sitio web se ha creado un mockup de las vistas a las que tendrá acceso el usuario con las rutas de estas.



El diseño fue realizado en la aplicación draw.io y puede ser cambiado fácilmente, el archivo se encuentra en la carpeta “DisenoNiveles”.

Base de datos (futuro):



Para una evolución sustancial del videojuego se ha preparado un pequeño diagrama en el que se representa las tablas de la base de datos en servidor que tendría.

En esta se ve representada la tabla “Usuario” en la que su clave principal se pasaría a la tabla “Partidas” cada vez que se creara una fila nueva, ya que las partidas deben estar asociadas obligatoriamente a un usuario que la haya jugado.

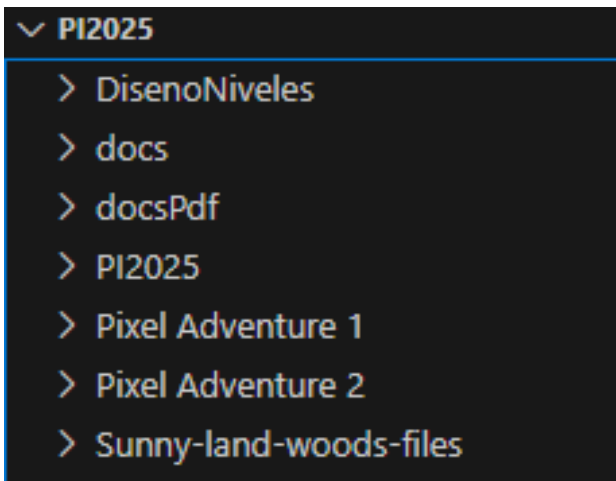
El diseño fue realizado en la aplicación draw.io y puede ser cambiado fácilmente, el archivo se encuentra en la carpeta “DisenoNiveles”.

Manual técnico

Este manual está dirigido a desarrolladores y técnicos encargados del mantenimiento, despliegue y evolución del videojuego. A continuación, se detallan los aspectos técnicos más relevantes de la aplicación, así como los procedimientos de instalación y mantenimiento del sitio web donde se aloja el juego.

ESTRUCTURA DEL PROYECTO

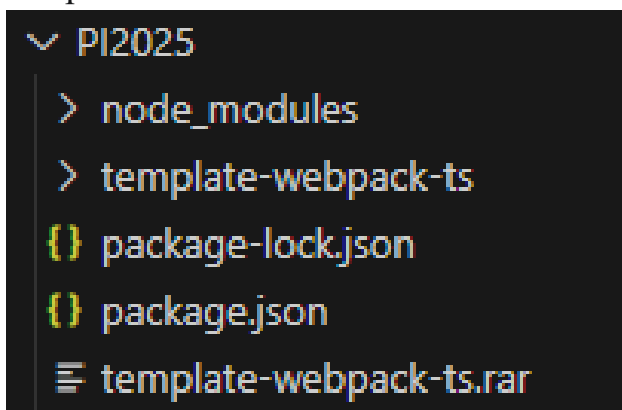
El proyecto está organizado de forma modular para facilitar su mantenimiento y escalabilidad. La estructura de carpetas es la siguiente:



Este es el apartado general en el encontramos las siguientes carpetas con su contenido:

- DisenoNiveles = creada para almacenar los bocetos de los niveles diseñados en formato digital..
- docs = en esta carpeta se encuentra la versión “build” del código del proyecto, es decir, es la carpeta que ayuda al despliegue de la aplicación de forma automatizada.
- docsPdf = almacena la documentación del proyecto, manual de usuario, manual técnico, documentación global y anteproyecto.
- PI2025 = almacena el código del proyecto, versión compilada y de desarrollo, además de archivo .zip/.rar
- Pixel Adventure 1 / Pixel Adventure 2 / Sunny-land-woods-files = almacena sprites de uso libre, algunos de ellos utilizados en el videojuego.

El apartado técnico se encuentra dentro de la carpeta “PI2025”:

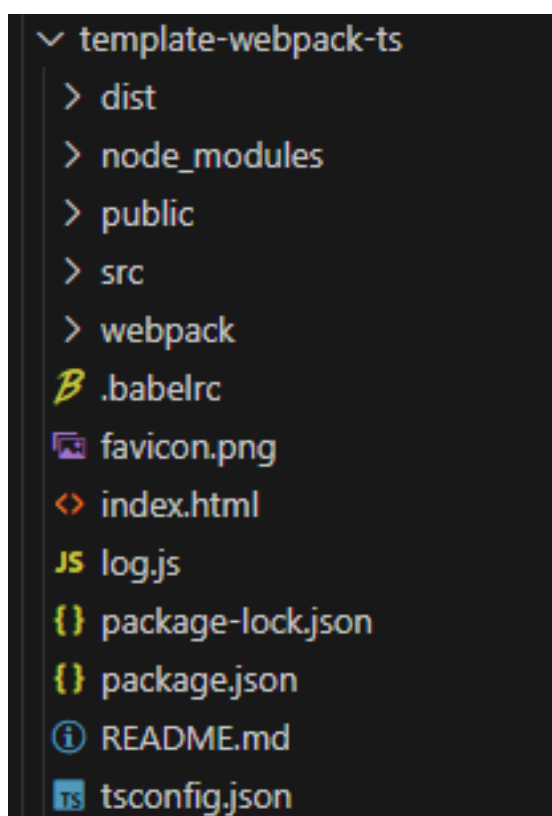


En esta carpeta encontramos:

- node_modules = almacena las dependencias necesarias para que el proyecto funcione.

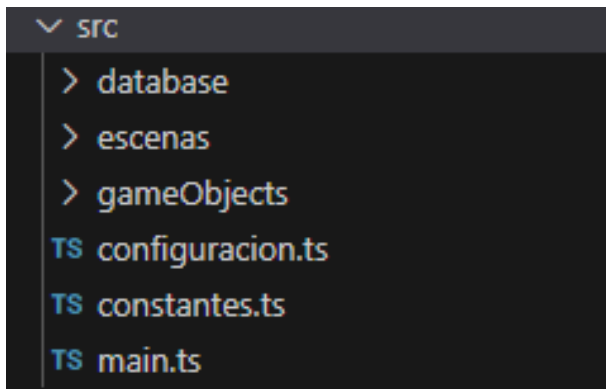
- template-webpack-ts = almacena el código del juego, compilado y la versión de desarrollo.
- package-lock.json / package.json = archivos de dependencias y configuración.

Dentro de la carpeta “template-webpack-ts” encontramos:



- dist = código compilado, listo para ser copiado y pegado en la carpeta docs para el despliegue automático.
- public = carpeta donde se encuentran todos los recursos visuales y sonoros del proyecto que se utilizan así como los archivos JSON que generan los niveles.
- src = almacena todo el código para el desarrollo del videojuego.
- index.html = archivo principal que es el núcleo de la aplicación. Este archivo es donde se ejecuta todo.

La carpeta “src” está compuesta por los siguientes elementos:



- database = carpeta que incluye la clase manejadora de la base de datos de cada navegador.
- escenas = carpeta que incluye todas las escenas del videojuego.
- gameObjects = carpeta que incluye los objetos interactuables del videojuego.
- configuraci3n.ts = Archivo que se encarga de la correcta configuraci3n del videojuego a trav3s de la importaci3n del objeto "Phaser.Types.Core.GameConfig". En esta configuraci3n se establece escala, escenas, render, f3sicas, audio y padre en el html principal.
- constantes.ts = En este archivo se declaran todas las constantes utilizadas en el resto del c3digo.
- main.ts = Clase que al ejecutarse comienza con la ejecuci3n de todo el juego. Es la clase principal.

Dicho esto ya hemos recorrido las partes esenciales de enrutamiento del proyecto. Ahora veremos las funciones principales.

FUNCIONES PRINCIPALES

En este apartado vamos a repasar las funciones principales que tiene este proyecto, como cambiarlas y/o implementar otras. Se ver3n las funciones b3sicas y explicaci3n de estas para el correcto entendimiento.

Este proyecto est3 creado con el entorno de ejecuci3n node.js, que permite a los desarrolladores crear aplicaciones del lado del servidor y herramientas de l3nea de comandos utilizando JavaScript, en su versi3n 22.14.0. Esta herramienta nos permite poder ejecutar el proyecto de manera sencilla y limpia, adem3s de hacer de gestor3a de dependencias.

Para la ejecución del proyecto necesitamos en la carpeta raíz un archivo index.html, este será encargado de llamar a la lógica de nuestra aplicación, concretamente al script en formato JavaScript.

¿Cómo llama a un script en formato JavaScript si estamos programando en el lenguaje TypeScript?

En este proyecto también se hace uso del framework de Phaser, este framework nos permite traer de manera sencilla una serie de elementos que nos ayuden a la hora de crear y diseñar un videojuego, pero además también funciona como transpilador para que automáticamente a la hora de la ejecución nuestro código TypeScript se transforme en un código JavaScript con las mismas funciones.

Este script ejecuta la clase Main, en la que simplemente carga una instancia de Game con la configuración que le pasamos en formato de constante a través de otro archivo.

Este archivo configuración tenemos lo siguiente:

```
export const config: Phaser.Types.Core.GameConfig = {
  type: Phaser.AUTO,
  scale: {
    mode: Phaser.Scale.FIT,
    autoCenter: Phaser.Scale.CENTER_BOTH,
    width: 854,
    height: 480,
    parent: 'game-container',
  },
  scene: [Load,
    Menu,
    HUD,
    Settings,
    Credits,
    LevelSelection,
    LevelEnd,
    Level01,
    Level02,
    Level03,
    Level04,
    Level05],
  render: {
    pixelArt : true,
    antialias:true
  },
  physics:{
    default:'arcade',
    arcade:{
      gravity:{x:0,y:600},
      debug:false
    }
  },
  audio: {
    disableWebAudio: true
  },
};
```

Tenemos una constante “config” del tipo de configuración de Phaser, y en ella se encuentran los siguientes elementos configurables:

- Scale: Escala del videojuego, su centrado y si tiene padre en html.
- Scene: Lista en la que se encuentran todas las escenas del videojuego.
- Render: Activación de elementos pixel art y antialias para la mejoría de rendimiento.
- Physics: Tipo de físicas, gravedad y modo debug para ver colisiones.
- Audio: para que se pueda habilitar o deshabilitar el audio.

Tras la carga de estos archivos de configuración y su puesta en uso, lo siguiente es cargar todos los elementos que necesita el videojuego para su correcto funcionamiento, sprites, sonido, niveles... Para ello el siguiente paso es ir a la escena “Load” la cual se encarga de hacer esto mismo.

Antes de entrar en las cargas de la escena “Load” veremos un ejemplo básico de una escena creada con Phaser.

EJEMPLO:

```
class EscenaPrincipal extends Phaser.Scene {
  constructor() {
    super({ key: 'EscenaPrincipal' });
  }
  preload() {
    // Cargar recursos (imágenes, sonidos, etc.)
    this.load.image('fondo', 'assets/fondo.png');
    this.load.image('jugador', 'assets/jugador.png');
  }
  create() {
    // Agregar fondo
    this.add.image(400, 300, 'fondo');
    // Agregar jugador
    this.jugador = this.add.sprite(400, 300, 'jugador');
    // Mostrar texto
    this.add.text(20, 20, '¡Bienvenido a la escena!', {
      font: '20px Arial',
      fill: '#ffffff'
    });
  }
  update(time, delta) {
```



```

    // Lógica de actualización por frame
    this.jugador.x += 1; // Mueve el jugador hacia la derecha
  }
}

```

En este ejemplo vemos, que la clase extiende de Phaser.Scene que es la parte de Phaser que nos ayuda a localizar esta clase como algo físico.

En el constructor usamos una key para dar nombre a la escena y poder referirnos a ella.

Método preload: Se encargar de cargar elementos antes de crear la escena.

Método create: Elementos que se crean inmediatamente después de crear la escena.

Método update: Lógica que necesita tener o tiene un constante cambio durante la escena.

Habiendo puesto esto como ejemplo veamos la clase “Load”.

Establecemos la ruta donde se encuentran los assets, color de fondo de la aplicación, creamos texto de “Cargando...” y establecemos unas barras. Estas barras su función es enseñar al usuario que el videojuego está cargando correctamente si avanza o incorrectamente si no avanza haciéndolo de manera visual.

Y hecho esto procede a cargar todos los assets que se utilizarán en el videojuego, sprites, mapas de niveles, sonidos...

La carga tiene la siguiente función:

```

this.load.image(Constants.MAPS.TILESET, 'levels/nivelestileset.png');

```

En la parte izquierda se necesita una palabra clave como key enlazado a esta imagen, en este caso, y a la derecha la ruta en la que se encuentra la imagen que queremos cargar.

Se ha utilizado el uso de constantes durante todo el desarrollo del videojuego para generar solidez en la programación y cambio de variables si fuese necesario.

Luego de la carga de los assets termina nuestro paso por la clase “Load” no sin antes establecer las variables globales de sonido a true y empezar la escena del menú.

```

this.scene.start(Constants.SCENES.MENU);

```

Con este método Phaser nos permite iniciar la escena menú, en este caso, sobre la escena de carga.

La escena “Menu” es muy básica ya que podemos decir que es una escena puente entre las demás escenas de nuestro proyecto. Esto lo que hace es que no tengamos gran funcionalidad más allá del cambio de escenas.

```
preload() {  
  this.createAndPlayMusic();  
}  
  
create() {  
  this.createBackground();  
  this.createTexts();  
  this.input.once("pointerdown", () => {  
    this.sound.unlock(); // Desbloquea el sistema de sonido en móviles  
  });  
}
```

El método `createAndPlayMusic` establece la instancia de la base de datos `localManager` del navegador, verifica el estado de la variable de música y carga el sonido.

El método `createBackground` establece el color de fondo para la escena.

El método `createTexts` crea todos los textos necesarios visuales en el menú. Para la creación de un texto tenemos los siguientes métodos:

```
createPlayText() {  
  const playTxtShadow = this.add  
    .bitmapText(  
      this.width / 2,  
      this.height / 1.1,  
      Constants.FONTS.BITMAP,  
      Constants.TEXTS.MENU.PLAY,  
      35  
    )  
    .setTint(0x000000); // Color negro para el borde  
  
  // Desplazar el texto de sombra para simular un borde  
  playTxtShadow.setOrigin(0.5, 0.5);  
  playTxtShadow.x -= 2;  
  playTxtShadow.y -= 2;  
  
  const playTxt: Phaser.GameObjects.BitmapText = this.add  
    .bitmapText(  
      this.width / 2,  
      this.height / 1.1,  
      Constants.FONTS.BITMAP,  
      Constants.TEXTS.MENU.PLAY,  
      35  
    )  
    .setInteractive();  
  
  playTxt.setOrigin(0.5, 0.5);  
  this.changeSceneToLevel(playTxt, Constants.SCENES.LEVEL_SELECTION);  
}
```

Todos los textos del videojuego se han establecido con `bitmapText` para hacer uso de la fuente `BITMAP`, añade la posición del texto, la fuente, el texto que se quiera mostrar y luego el tamaño, más tarde se da color al texto con el método `.setTint()`. Con `.setOrigin()` decimos el lugar de origen del texto y más tarde se establece `this.changeSceneToLevel()` como primer elemento el texto que hay que clicar y como segundo elemento el nombre de la escena a la que se dirige., veamos que hace este método.

```
changeSceneToLevel(playTxt: Phaser.GameObjects.BitmapText, scene: string) {
  playTxt.on("pointerdown", () => {
    this.scene.start(scene); // Lanza la escena LevelSelection y detiene la actual
  });
}

changeSceneToSettings(
  settingsTxt: Phaser.GameObjects.BitmapText,
  scene: string
) {
  settingsTxt.on("pointerdown", () => {
    this.scene.start(scene); // Lanza la escena Settings sin detener la actual
  });
}
```

En los métodos de cargar la escena vemos lo siguiente:

`Texto.on("pointerdown"` = Esto hace referencia a cuando se clicca en este texto sucederá la función pasada a través del arrow function.

Este arrow function inicia la escena pasada por parámetro.

Desde el menú se puede navegar al selector de niveles, ajustes o créditos. Primero vamos a ver la escena de Créditos.

La escena de Créditos carga textos y una imagen central del logo del juego, en los textos se cargan las constantes que indican de donde se han recogido los sprites y sonidos utilizados.

Además por último hay un botón de volver en el que se inicia el mismo método que hemos visto anteriormente para poder volver al menú.

Ahora veremos la escena de Ajustes.

En esta escena se cargan, además de los textos, dinámicamente las imágenes de sonido, según la variable en la base de datos cargará la fuente de sonido como si estuviera activa y viceversa.

```

let musicOnOff: Phaser.GameObjects.Image = this.add
    .image(
        this.cameras.main.width / 3,
        this.cameras.main.height / 2.5,
        this.getSoundImg(database.data.music)
    )
    .setScale(0.1)
    .setInteractive();
let effectsOnOff: Phaser.GameObjects.Image = this.add
    .image(
        this.cameras.main.width / 3,
        this.cameras.main.height / 1.6,
        this.getSoundImg(database.data.effects)
    )
    .setScale(0.1)
    .setInteractive();

```

La carga dinámica se hace a través del método `getSoundImg`.

```

getSoundImg(sound:boolean): string {
    return sound ? Constants.SETTINGS.SOUND_ON : Constants.SETTINGS.SOUND_OFF
}

```

Este método devuelve, según el booleano de la base de datos, la constante que se refiere al sonido activado o desactivado.

Al haberle dado interactividad a las imágenes la utilizamos de la siguiente manera:

```

musicOnOff.on('pointerdown', () => {
    database.data.music = !database.data.music;
    database.saveDB();
    musicOnOff.setTexture(this.getSoundImg(database.data.music));

    if(!database.data.music){
        this.backgroundMusic.stop();
    }else{
        this.backgroundMusic.play();
    }
});
effectsOnOff.on('pointerdown', () => {
    database.data.effects = !database.data.effects;
    database.saveDB();
    effectsOnOff.setTexture(this.getSoundImg(database.data.effects));
});

```

Con esto hacemos referencia a que cuando una imagen sea clicada, cambiará la variable en la base de datos, la guardará y en el caso de la música procederá a pararla o activarla, según sea el caso.

En esta escena también tenemos un botón de volver que nos devuelve a la escena menú.

Ahora veremos la clase de selección de niveles.

En esta clase también tenemos una funcionalidad muy sencilla siendo la misma que en las demás pantallas, esta pantalla también actúa de puente entre escenas, en este caso, entre las escenas de los niveles con la del menú. Por ello esta escena carga textos con interactividad con la que iniciar la escena de los niveles.

En esta escena también tenemos un botón de volver que nos devuelve a la escena menú.

```
changeSceneToLevel(playTxt: Phaser.GameObjects.BitmapText, scene: string) {  
  playTxt.on('pointerdown', () => {  
    playTxt.disableInteractive();  
    this.cameras.main.fade(500, 0, 0, 0);  
    this.cameras.main.once('camerafadeoutcomplete', () => {  
      this.sound.stopAll();  
      this.scene.start(scene);  
      this.scene.start(Constants.SCENES.HUD, {levelName: scene});  
      this.scene.bringToTop(Constants.SCENES.HUD);  
    });  
  });  
}
```

En este método se establece la interactividad con cada texto que se refiere a un nivel. Cuando el usuario clicca en un elemento con esta interactividad sucede lo siguiente: Le inhabilita la interactividad al elemento en cuestión para que no clique más veces y luego empieza un fundido a negro para empezar la escena, cuando este fundido es completado para todos los sonidos y empieza la escena en cuestión. Además también empieza de forma sincronizada la escena HUD que repasaremos en el siguiente punto, esta escena HUD se establece con el método bringToTop para que sea la escena más cercana a la pantalla, es decir, esta escena se verá por encima de la escena de nivel.

FUNCIONES DE NIVELES

En este punto vamos a repasar las funciones de niveles, siendo estas la interfaz gráfica de información de usuario, la información que se crea a través de los niveles, que se recoge, muestra y que se guarda.

Para ello vamos a ver primeramente la clase HUD:

En esta clase tenemos información que se mostrará al usuario en tiempo real sobre el tiempo restante que le queda para superar el nivel además de las vidas y puntuación que tiene. Para ello necesitamos crear los textos de vidas, y puntuación.

Estos textos se actualizarán de esta manera:

```
private updateLives(): void {  
    this.livesTxt.text =  
        "VIDAS:" + this.registry.get(Constants.GLOBAL_VARIABLES.LIFES);  
}  
private updateScore(): void {  
    this.scoreTxt.text =  
        "SCORE:" + this.registry.get(Constants.GLOBAL_VARIABLES.SCORE);  
}  
  
private updateTime(): void {  
    this.timeTxt.text = this.registry.get(Constants.GLOBAL_VARIABLES.TIME);  
}
```

Si sucede algo que hace que cambie el registro de vidas, puntuación o tiempo, actualizará directamente el texto al que van asociadas por lo que el usuario podrá ser visualizador en tiempo real de esta información.

Además se añade que, si el dispositivo que está corriendo el juego es táctil, se crearán unos controles en la pantalla para que el usuario pueda jugar también desde dispositivos móviles.

```
if (this.sys.game.device.input.touch) {  
    this.createHudControls();  
}
```

A través del create definimos esta condición que nos lleva al método createHudControls.

```

createHudControls() {
    this.input.addPointer(2);

    const controlSize = 100;
    const padding = 1;
    const scaleFactor = 0.15;

    const baseXRight = this.cameras.main.width - controlSize - padding;
    const baseY = this.cameras.main.height - controlSize - padding;

    this.leftControl = this.add
        .sprite(
            baseXRight - controlSize - padding,
            baseY,
            Constants.CONTROLS.LEFT
        )
        .setOrigin(0.5, 0.5)
        .setInteractive()
        .setScale(scaleFactor);

    this.rightControl = this.add
        .sprite(baseXRight, baseY, Constants.CONTROLS.RIGHT)
        .setOrigin(0.5, 0.5)
        .setInteractive()
        .setScale(scaleFactor);

    const baseXLeft = controlSize + padding;
    this.upControl = this.add
        .sprite(baseXLeft, baseY, Constants.CONTROLS.UP)
        .setOrigin(0.5, 0.5)
        .setInteractive()
        .setScale(scaleFactor);

    this.leftControl.on("pointerdown", () => {
        this.level.player.leftControl = true;
    });
    this.leftControl.on("pointerup", () => {
        this.level.player.leftControl = false;
    });

    this.rightControl.on("pointerdown", () => {
        this.level.player.rightControl = true;
    });
    this.rightControl.on("pointerup", () => {
        this.level.player.rightControl = false;
    });

    this.upControl.on("pointerdown", () => {
        this.level.player.upControl = true;
    });
    this.upControl.on("pointerup", () => {
        this.level.player.upControl = false;
    });

    const controlContainer = this.add.container(0, 0);
    controlContainer.add([this.leftControl, this.rightControl, this.upControl]);
    controlContainer.setAlpha(0.8).setScrollFactor(0).setDepth(5);
}

```

Con estas definiciones creamos las imágenes y le damos interactividad en un array para que pueda ser mostrado. Los pointerdown se refieren a la acción cuando pulsas un botón y el pointerup es la acción de cuando dejas de pulsarlo.

Para mejorar la experiencia de usuario hemos puesto los iconos de desplazamiento lateral en la parte derecha y el desplazamiento vertical en la parte izquierda, de modo que no tenga todos los controles para una sola mano ya que puede ser incómodo si necesita saltar e ir hacia un lateral de manera sincronizada.

Esta clase se muestra por encima del nivel en cuestión para que se pueda visualizar siempre.

Ahora vamos a ver la clase manejadora de niveles.

Para la correcta modularización del proyecto hemos creado la clase “LevelHandler” que actúa de padre de todos los niveles que se creen para así evitar la repetición de código de manera innecesaria y asegurar una implementación correcta de más niveles si fuera necesario.

```
export default class LevelHandler extends Phaser.Scene{
```

Esta clase extiende de Phaser.Scene al igual que las escenas anteriores, al ser padre de las escenas de niveles, que veremos más tarde, se heredarán las propiedades de Phaser.Scene además de las de la clase “LevelHandler”.

Esta clase es la encargada de crear el conjunto de: jugador, enemigos, coleccionables y plataformas móviles dispuestas en el nivel.

Cada uno de los niveles han sido creados a través de la aplicación Tiled. Esta aplicación genera un JSON con la información de los elementos que se han utilizado. Estos JSON los podemos encontrar en la ruta /public/assets/levels/ dentro de la carpeta template-webpack-ts para su implementación.

En la inicialización del LevelHandler tenemos lo siguiente:

Inicialización de vidas, puntuación y tiempo en segundos.

Para todos los sonidos, (para luego establecer la canción dedicada a los niveles).

Registra las variables en el registro global.

Crea un array de enemigos.


```

init():void{
    this.lives = 3;
    this.score = 0;

    this.seconds = 1;
    this.timeRemaining = 240; // 4 minutos
    this.timeDone = false;
    this.sound.stopAll();

    this.registry.set(Constants.GLOBAL_VARIABLES.LIFES, this.lives);
    this.registry.set(Constants.GLOBAL_VARIABLES.SCORE, this.score);

    this.enemyGroup = [];
}

```

Durante el método preload():

Instancia la base de datos y si la variable de música es true, inicia la música dedicada a los niveles.

Ahora se define el método createStage:

```

createStage(jsonMap:string,platformLayer:string,background:string):void{
    this.createMap(jsonMap,platformLayer);
    this.createBackgroundScroll(background);
    this.createAnimations();
    this.createPlayer();
    this.createObjects();
    this.createPlatforms();
}

```

Este es el método que se encarga de generar el mapa, animaciones, enemigos, jugador, objetos y plataformas. Los parámetros que se pasan deben ser las referencias al mapa JSON, a la capa de plataformas utilizada, y el fondo que se utilizará en el nivel.

En el método createMap cargamos el tileset del mapa, estableciendo la medida en pixeles. A través del this.physics.world.bounds.setTo() establecemos los bordes del mapa para que el jugador no pueda traspasarlos.

Si el tileset no carga correctamente se lanzará un error para parar la ejecución.

Por último generamos los bordes de colisiones para las plataformas, así el usuario no las traspasará.

```

createMap(jsonMap:string,platformLayer:string){
    // Cargamos el mapa y la capa de plataformas
    this.mapaNivel = this.make.tilemap({key: jsonMap, tileWidth:16,tileHeight:16});
    this.physics.world.bounds.setTo(0,0,
        this.mapaNivel.widthInPixels,this.mapaNivel.heightInPixels
    );

    if (this.mapaNivel.addTilesetImage(Constants.MAPS.TILESET)==null) {
        throw new Error(`No se pudo cargar el tileset ${Constants.MAPS.TILESET}`);
    }else{
        this.conjuntoPatrones = this.mapaNivel.addTilesetImage(Constants.MAPS.TILESET)!;
    }

    this.capaMapaNivel = this.mapaNivel.createLayer(platformLayer, this.conjuntoPatrones)!;
    this.capaMapaNivel.setCollisionByExclusion([-1]);
}

```

El método createBackgroundScroll es el encargado de cargar el fondo, hacer que se repita y además darle movimiento.

```

createBackgroundScroll(background:string):void{
    this.backgroundImage = this.add.tileSprite(0,0,this.mapaNivel.widthInPixels,
        this.mapaNivel.heightInPixels,background).setOrigin(0,0).setDepth(-1);

    this.levelBackgroundName = background;
}

```

El método createAnimations es el que se encarga de crear las animaciones usando datos de JSON o una imagen con recorrido.

Para ello utiliza el this.anims.create:

```

this.anims.create({
    key: Constants.PLAYER.ANIMATION.IDLE,
    frames: this.anims.generateFrameNames(Constants.PLAYER.ID, {
        prefix: Constants.PLAYER.ANIMATION.IDLE + '-', // Prefijo usado en los nombres de los frames
        start: 0, // Empieza desde el primer frame en el JSON para 'idle'
        end: 10 // Último número de frame de 'idle' en el JSON
    }),
    frameRate: 20,
    repeat: -1 // Repetir indefinidamente
});

```

Este create usa datos de un JSON por lo que debemos utilizar generateFrameNames para poder crearlos a través de este sin tener ningún problema.

La otra forma de poder crear una animación es la siguiente:

```
// ANIMACION MUERTE ENEMIGO
this.anims.create({
  key: Constants.ENEMIES.DEATH.ANIM,
  frames: Constants.ENEMIES.DEATH.ID,
  frameRate: 15,
  repeat: 0
});
```

De esta manera únicamente tenemos que poner la clave con la que la llamaremos, el nombre de los frames (que al no ser un JSON sino una imagen estática dividida no hace falta hacer un generateFrameNames, frameRate de la animación y si queremos que se repita x veces. Con esto abríamos creado las animaciones:

- Jugador parado
- Jugador corriendo
- Muerte de enemigo
- Frutas
- Recolección de frutas

Ahora vemos el método createPlayer:

```
createPlayer():void{
  // Busca si hay un objeto que se llame como el id en el mapa.
  let objeto = this.mapaNivel.findObject(Constants.PLAYER.ID, (d: any) => {
    return d.name === Constants.PLAYER.ID; // Suponiendo que estás buscando por el nombre del objeto
  });

  if (objeto) {
    this.player = new Player({
      escena: this,
      x: objeto.x,
      y: objeto.y,
      texture: Constants.PLAYER.ID
    });
    this.physics.add.collider(this.player, this.capaMapaNivel);

    this.cameras.main.setBounds(0, 0, this.mapaNivel.widthInPixels, this.mapaNivel.heightInPixels);
    this.cameras.main.startFollow(this.player);
  }
}
```

Buscamos en el mapa del nivel un objeto que se llame como el ID del jugador definido en constantes. Luego, si encontramos el objeto lo establecemos como clase Jugador, en una posición y con la textura asociada al ID del jugador. Le añadimos colisión, además de los bordes de mapa y hacemos que la cámara siga al jugador.

Más tarde veremos la clase “Player”, atributos y métodos.

El método createObjects hace lo siguiente:

```
createObjects():void{
    let finalObject: any = this.mapaNivel.createFromObjects(
        Constants.MAPS.FINAL_POSITION,
        {name:Constants.MAPS.FINAL_POSITION})[0];

    this.physics.world.enable(finalObject);
    finalObject.body.setAllowGravity(false);
    finalObject.setTexture(Constants.OBJECTS.FINAL_OBJECT);
    finalObject.body.setSize(40,70);
    finalObject.body.setImmovable(true);

    this.physics.add.collider(this.player, finalObject, ()=>{
        this.stopLevel();
    });
}
```

Es el encargado de crear el objeto de la copa, con el que el jugador ganará el nivel cuando lo toque. Lo busca en el mapa nivel, lo habilita, le quita la gravedad y establece que no se pueda mover, le genera la textura del objeto final y además le añade colisión. Si esta colisión ocurre, llama al método stopLevel.

```
stopLevel():void {
    let isWin: boolean = false;

    if(this.lives > 0 || this.timeDone){
        isWin = true;
    }

    this.cameras.main.fade(200, 0, 0, 0);
    this.cameras.main.once('camerafadeoutcomplete', () => {
        this.stopAndSendInfoLevelEnd(isWin);
    });
}
```

El método stopLevel es el encargado de ver por qué termina el nivel y que solución se manda. Si el jugador tiene más de 0 vidas y el tiempo aún no ha terminado se

mandará como ganado al final del nivel a través del método `stopAndSendInfoLevelEnd`, en caso contrario se mandará como que ha perdido.

El método `stopAndSendInfoLevelEnd`:

```
stopAndSendInfoLevelEnd(isWin: boolean): void {
    this.sound.stopAll();
    this.scene.stop(this.levelName);
    this.scene.stop(Constants.SCENES.HUD);
    this.scene.start(Constants.SCENES.LEVEL_END, {
        isWin: isWin,
        levelName: this.levelName,
        score: this.score + this.timeRemaining,
        levelBackgroundName: this.levelBackgroundName
    });
}
```

Para las escenas e inicia la escena de “LevelEnd” mandando si ha ganado, puntuación y nombre del nivel.

Método `createCollectables` y `createEnemies`:

```
createCollectables(collectablesConfig:any[]):void{
    collectablesConfig.forEach((collectable) => {
        let collectables : Collectables = new Collectables(
            this,
            Constants.MAPS.COLLECTABLES,
            collectable.ID,
            collectable.ANIMATION
        );
        this.physics.add.overlap(this.player, collectables, this.player.collectObject as Phaser.Types.Physics.Arcade.ArcadePhysicsCallback, undefined, this);
    });
}

createEnemies(enemyConfig:any[]): void {
    enemyConfig.forEach((enemy) => {
        let enemies : Enemies = new Enemies(
            this,
            Constants.MAPS.ENEMIES,
            enemy.ID,
            enemy.ANIMATION,
            enemy.VELOCITY
        );
        this.physics.add.collider(enemies, this.capaMapaNivel);
        this.physics.add.overlap(this.player, enemies, this.player.enemyCollision as Phaser.Types.Physics.Arcade.ArcadePhysicsCallback, undefined, this);
        this.enemyGroup.push(enemies);
    });
}
```

Estos método se establecen a raíz de un array, se busca en el mapa nivel todos los enemigos y frutas, y según su nombre se establece id, animación y velocidad en caso de los enemigos. Luego se añade una colisión.

El método `createPlatforms` es el encargado de crear las plataformas que se mueven y añadirles colisión. Lo hace a través de la clase “Platforms” que veremos más tarde.

El método `createDecorativeImages` crea a través de los objetos del tilemap del nivel, si hay imágenes las establece.

El método createLevelText establece los textos de la misma manera que anteriormente las imágenes.

En el método update del nivel ocurre lo siguiente:

```
update(time: number): void {
    // Mover fondo
    this.backgroundImage.tilePositionY -= 0.4;

    this.player.update();

    this.enemyGroup.forEach((enemy) => {
        enemy.update();
    });
    this.platformsGroupH.update();
    this.platformsGroupV.update();

    if(this.lives <= 0){
        this.stopLevel();
    }

    // Gestión del tiempo
    if((this.seconds != Math.floor(Math.abs(time/1000)))&& !this.timeDone){
        this.seconds = Math.floor(Math.abs(time/1000));
        this.timeRemaining--;

        let minutes: number = Math.floor(this.timeRemaining/60);
        let segundos: number = Math.floor(this.timeRemaining - (minutes*60));

        let timeText:string = Phaser.Utils.String.Pad(minutes,2,'0',1) + ":" + Phaser.Utils.String.Pad(segundos,2,'0',1);
        this.registry.set(Constants.GLOBAL_VARIABLES.TIME,timeText);
        this.events.emit(Constants.EVENTS.TIME);

        if(this.timeRemaining == 0) {
            this.timeDone= true;
            this.stopLevel();
        }
    }
    if(this.player.y + this.player.height / 2 >= this.physics.world.bounds.bottom){
        this.player.worldBoundCollision(this.player);
    }
}
```

Se llama al update de jugador, enemigos y plataformas. Está revisando si las vidas son menores o igual a cero para finalizar nivel en caso de que lo sea. También revisa el tiempo restante y lo actualiza, además de llamar a los cambios de registros. Finalizando revisa que el jugador no colisione con la parte inferior de la pantalla destinada al juego, y si lo hace llama al método worldBoundCollision del jugador para hacer daño y quitarle una vida.

Por último, en este apartado vamos a ver la clase “LevelEnd”.

Esta clase se encarga de manejar lo necesario al terminar un nivel, ya haya el jugado ganado o perdido.

Empezando, se inician las variables según el dato que hay llegado desde el nivel como hemos visto anteriormente.

Luego si el jugador ha ganado se crea y muestra el texto de que ha ganado, con la puntuación. Además, guarda esta información en la base de datos llamando al método saveDB.

```
if(this.database.data.levels[level].score < this.score){
    this.database.data.levels[level].score = this.score;
    this.database.data.levels[level].isPassed = true;
    this.database.saveDB();
}
```

También se crea el botón de volver al menú con interactividad para ello.

Como no es necesario hacerlo de manera completa, vamos a ver a continuación un ejemplo de clase de nivel. El ejemplo será la clase de “Level05”.

```
import LevelHandler from "../levelHandler";
import Constants from "../../constantes";

export class Level05 extends LevelHandler {
    constructor() {
        super(Constants.SCENES.LEVELS.LEVEL_5);
    }

    create(): void {
        this.createStage(
            Constants.MAPS.LEVELS.LEVEL_5.TILEMAP_JSON,
            Constants.MAPS.PLATFORM_LAYER,
            Constants.BACKGROUNDS.LEVEL_4
        );

        this.createEnemies([Constants.ENEMIES.BUNNY]); // , Constants.ENEMIES.BEE para añadir más enemigos
        this.createCollectables([
            Constants.OBJECTS.COLLECTABLES.FRUIT.APPLE,
            Constants.OBJECTS.COLLECTABLES.FRUIT.BANANA,
            Constants.OBJECTS.COLLECTABLES.FRUIT.CHERRY,
        ]);

        this.createDecorativeImages(
            Constants.MAPS.FAKE_FINAL,
            Constants.OBJECTS.FINAL_OBJECT
        );
    }
}
```

Esta clase es muy sencilla ya que únicamente hace llamadas a los métodos de la clase “LevelHandler” con la información de cada nivel, en este caso vemos:

- Llama a createStage, con los valores del mapa 5.

- Llama a createEnemies con las clases de enemigos que existen en este mapa. Si hubiera más se añadirían como se indica en el comentario.
- Llama a createCollectables con los tipos de frutas que existen en el mapa.
- Llama a createDecorativeImages con las imágenes alternas necesarias para su implementación.
-

Por último en este apartado, vamos a ver la clase “DBManager”.

Esta clase es la encargada de manejar toda la información de la aplicación a través del localStorage del navegador. Es la que registra los cambios y a través de accesos a ella se verifican ciertos aspectos de la aplicación como la música, la puntuación de los niveles, si se ha pasado cierto nivel o no...

En el constructor de la clase encontramos lo siguiente:

```
constructor() {
  const dbData = localStorage.getItem(Constants.LOCAL_STORAGE.DB_NAME);
  if (dbData) {
    try {
      this.data = JSON.parse(dbData);
    } catch (error) {
      console.error("Error al parsear los datos de la base de datos:", error);
      this.createDB();
    }
  } else {
    this.createDB();
  }
}
```

Vemos si el navegador ya tiene un localStorage con la información.

Si lo tuviera, parseamos esta información a JSON para poder trabajar con ella. Si no fuera el caso creamos la información a través de createDB.

La información se crea con la siguiente estructura:

Music: variable de música.

Effects: variable de efectos de sonido.

Levels: todos los niveles con su puntuación y si se ha pasado o no.

Inicia los datos y guarda la base de datos.


```

private createDB(): void {
    const initialDB = {
        music: true,
        effects: true,
        levels: {
            level01: {
                score: 0,
                isPassed: false
            },
            level02: {
                score: 0,
                isPassed: false
            },
            level03: {
                score: 0,
                isPassed: false
            },
            level04: {
                score: 0,
                isPassed: false
            },
            level05: {
                score: 0,
                isPassed: false
            }
        }
    };
    this.data = initialDB;
    this.saveDB();
}

```

CLASES DE OBJETOS

En este apartado veremos las clases de objetos que hay y sus métodos.

Clase Player:

Empezando por la clase de “Player” esta es la encargada de dar movimiento, sonido y textura al jugador dentro del videojuego.

En el constructor de la clase tenemos lo siguiente:

```
constructor(config: any) {  
    super(config.escena, config.x, config.y, config.texture);  
  
    this.initialPositionX = config.x;  
    this.initialPositionY = config.y;  
    this.escena = config.escena;  
    this.escena.physics.world.enable(this);  
    this.escena.add.existing(this);  
  
    this.body?.setSize(20, 25);  
    this.setCollideWorldBounds(true);  
  
    this.movementByKeyboard();  
    this.animationSpritesPlayer();  
  
    this.collisionActive = false;  
    this.collectingActive = false;  
    this.jumpCount = 0;  
  
    this.jumpSound = this.escena.sound.add(Constants.SOUNDS.EFFECTS.JUMP, {  
        loop: false,  
        volume: 1.75  
    });  
  
    this.database = new DBManager();  
  
    this.play(Constants.PLAYER.ANIMATION.IDLE);  
}
```

Se establecen los valores y se le da un tamaño, establecemos que choque con los bordes del mapa.

A través del método `movementByKeyboard` hacemos que se pueda mover cuando se pulsa el teclado:

```
movementByKeyboard(): void {  
    // Entrada de movimiento por teclado  
    this.cursors = this.escena.input.keyboard?.createCursorKeys()!;  
    this.wasd = this.escena.input.keyboard?.addKeys('W,A,S,D');  
    this.space = this.escena.input.keyboard?.addKey(Phaser.Input.Keyboard.KeyCodes.SPACE)!;  
}
```

Concretamente las teclas “W,A,S,D,ESPACIO” y las flechas.

Llamamos al método `animationSpritesPlayer` para crear las animaciones.

Y luego creamos el sonido de salto a través de la palabra reservada. Instanciamos la base de datos y ejecutamos la animación “IDLE”.

En el método update de la clase:

```
update(): void {
    // Control de movimiento del jugador
    // Izquierda
    if (this.wasd.A.isDown || this.cursors.left.isDown || this.leftControl) {
        this.setVelocityX(-150);
        this.flipX = true;
        if (this.body?.blocked.down) {
            this.anims.play(Constants.PLAYER.ANIMATION.RUN, true);
        }

        // Derecha
    } else if (this.wasd.D.isDown || this.cursors.right.isDown || this.rightControl) {
        this.setVelocityX(+150);
        this.flipX = false;
        if (this.body?.blocked.down) {
            this.anims.play(Constants.PLAYER.ANIMATION.RUN, true);
        }

        // Quieto
    } else {
        this.setVelocityX(0);
        if (this.body?.blocked.down) {
            this.anims.play(Constants.PLAYER.ANIMATION.IDLE, true);
        }
    }
}
```

Estamos apuntando al observable del teclado y los controles táctiles. Si se pulsan se dará velocidad al jugador con la animación “RUN”.

Lo mismo ocurre con el salto, la diferencia es con el contador jumpCount. Este establece un límite máximo de saltos que se reestablece cuando el jugador toca el suelo. Así el jugador podrá hacer dobles saltos durante el transcurso del nivel.

También se establece la animación “FALL” cuando el jugador no está tocando el suelo y su velocidad hace que vaya hacia abajo.

```

// Salto
const jumpPressed =
    Phaser.Input.Keyboard.JustDown(this.space) ||
    Phaser.Input.Keyboard.JustDown(this.wasd.W) ||
    Phaser.Input.Keyboard.JustDown(this.cursors.up) ||
    this.upControl; // Asumiendo que upControl se activa solo al presionar, no mantener

// Ejecutar salto si aún no se alcanza el máximo permitido
if (jumpPressed && this.jumpCount < this.maxJumps - 1) {
    if (this.database.data.effects) {
        this.jumpSound.play();
    }

    this.setVelocityY(-250);
    this.anims.stop();
    this.setTexture(Constants.PLAYER.ID, Constants.PLAYER.ANIMATION.JUMP);
    this.jumpCount++;

    // Reiniciar el control táctil para evitar múltiples saltos por mantener presionado
    this.upControl = false;
}

if (this.body?.blocked.down) {
    this.jumpCount = 0;
}

// Caída
if (this.body && this.body.velocity && !this.body.blocked.down && this.body.velocity.y > 0) {
    this.setTexture(Constants.PLAYER.ID, Constants.PLAYER.ANIMATION.FALL);
}

```

El método `worldBoundsCollision` es el que establece que el jugador se hará daño si choca con el borde inferior de la pantalla de juego. Además lo hará retroceder a la posición de inicio del nivel perdiendo así su progreso.

```

public worldBoundsCollision(player: Player) {
    player.escena.lives--;
    player.escena.registry.set(Constants.GLOBAL_VARIABLES.LIFES, player.escena.lives);
    player.escena.events.emit(Constants.EVENTS.LIFES);
    player.tint = 0xff0000;
    let playerDamageSound: Phaser.Sound.BaseSound = player.escena.sound.add(Constants.SOUNDS.EFFECTS.PLAYER_DAMAGE, {
        loop: false,
        volume: 0.5
    });
    if (player.database.data.effects) {
        playerDamageSound.play();
    }
    player.escena.time.addEvent({
        delay: 600,
        callback: () => {
            player.collisionActive = false;
            player.clearTint();
        }
    });
    player.setPosition(player.initialPositionX, player.initialPositionY);
}

```

El método `enemyCollision` es el que se encarga de ver si el jugador colisiona con un objeto enemigo.

Para ello tenemos dos casos:

Cuando el jugador pisa al enemigo.

En este caso, al ser el jugador el que pisa al enemigo se le dará un rebote para que salga hacia arriba, se establecerá un sumatorio de puntos y además se carga y ejecuta el sonido de explosión de enemigo, además del Sprite para que el enemigo explote. Finalmente se elimina al enemigo de la pantalla de juego.

```
public enemyCollision(player: Player, enemy: Phaser.Physics.Arcade.Sprite): void {
    if (
        player.body &&
        enemy.body &&
        player.body.velocity.y > 1 &&
        enemy.body.touching.up &&
        player.body.touching.down
    ) {
        const bounceX = player.x < enemy.x ? -100 : 100;
        const bounceY = -200;
        player.setVelocity(bounceX, bounceY);

        if (!player.collisionActive) {
            let posX = enemy.x;
            let posY = enemy.y;
            enemy.destroy();

            player.escena.score += 100;
            player.escena.registry.set(Constants.GLOBAL_VARIABLES.SCORE, player.escena.score);
            player.escena.events.emit(Constants.EVENTS.SCORE);

            let explosion: Phaser.GameObjects.Sprite = player.escena.add.sprite(
                posX,
                posY,
                Constants.ENEMIES.DEATH.ID
            );

            let explosionSound: Phaser.Sound.BaseSound = player.escena.sound.add(Constants.SOUNDS.EFFECTS.ENEMY_DESTROY, {
                loop: false,
                volume: 0.7
            });
            if (player.database.data.effects) {
                explosionSound.play();
            }
            explosion.play(Constants.ENEMIES.DEATH.ANIM);
            explosion.once('animationcomplete', () => {
                explosion.destroy();
            });
        }
    }
}
```

El enemigo choca con el jugador en cualquier otro caso.

El jugador recibirá un rebote hacia un lateral impulsándolo lejos del enemigo. Se registrará un cambio en la variable de vida y tintará al jugador de color rojo. Sonará el efecto correspondiente a este y se añadirá un evento que tardará 0.6 segundos que hará que cuando acabe limpie el tintado del Sprite del jugador.

```

} else if (!player.collisionActive) {
  const bounceX = player.x < enemy.x ? -150 : 150;
  const bounceY = -250;
  player.setVelocity(bounceX, bounceY);

  player.escena.lives--;
  player.escena.registry.set(Constants.GLOBAL_VARIABLES.LIFES, player.escena.lives);
  player.escena.events.emit(Constants.EVENTS.LIFES);

  player.collisionActive = true;
  player.tint = 0xff0000;

  let playerDamageSound: Phaser.Sound.BaseSound = player.escena.sound.add(Constants.SOUNDS.EFFECTS.PLAYER_DAMAGE, {
    loop: false,
    volume: 0.5
  });
  if (player.database.data.effects) {
    playerDamageSound.play();
  }

  player.escena.time.addEvent({
    delay: 600,
    callback: () => {
      player.collisionActive = false;
      player.clearTint();
    }
  });
}
}

```

El método collectObject se encarga de manejar cuando el jugador recolecta una fruta.

```

public collectObject(player: Player, collectable: Phaser.Physics.Arcade.Sprite): void {
  if (!this.collectingActive) {
    this.collectingActive = true;

    player.escena.score += 50;
    player.escena.registry.set(Constants.GLOBAL_VARIABLES.SCORE, player.escena.score);
    player.escena.events.emit(Constants.EVENTS.SCORE);

    let collectSound: Phaser.Sound.BaseSound = player.escena.sound.add(Constants.SOUNDS.EFFECTS.COLLECT, {
      loop: false,
      volume: 1
    });
    if (player.database.data.effects) {
      collectSound.play();
    }

    collectable.play(Constants.OBJECTS.ANIMATION);
    collectable.once('animationcomplete', () => {
      collectable.destroy();
      this.collectingActive = false;
    });
  }
}

```

Para ello necesita emitir un registro en la puntuación y además es el encargado de ejecutar el efecto de recolección de objetos. Por último, elimina la fruta recolectada.

Estos dos últimos métodos tienen por parámetros tanto el enemigo y recolectable que es parte de la acción, como el jugador. Esto es por que estos métodos no son llamados directamente desde el jugador, sino que es el manejador de niveles el

encargado de llamarlos en el caso que sea necesario aún teniendo un impacto directo en el jugador.

Clase Enemies:

Ahora vamos a ver la clase “Enemies” referente a los enemigos del juego.

Esta clase extiende de “Phaser.Physics.Arcade.Group” ya que no va a ser un objeto individual sino un grupo de objetos.

En el constructor iniciamos el nivel al cual pertenece, nombre, id, animación y velocidad del enemigo, se le da las físicas a cada enemigo de forma individual y se selecciona de manera aleatoria la dirección a la que van.

```
moveEnemy(direction: string, enemy: any): void {
  if (direction === 'left') {
    enemy.body.setVelocityX(-this.velocity);
    enemy.flipX = false; // Voltea el sprite a la izquierda
  } else if (direction === 'right') {
    enemy.body.setVelocityX(this.velocity);
    enemy.flipX = true; // Voltea el sprite a la derecha
  }
}

public update(): void{
  this.children.entries.map((enemy:any) => {
    if(enemy.body.velocity.x === 0){
      this.moveEnemy(Phaser.Math.Between(0,1) ? 'left' : 'right', enemy);
    }
    if(enemy.body.blocked.right || enemy.body.blocked.left) {
      if (enemy.body.velocity.x < 0) {
        this.moveEnemy('right', enemy);
      } else {
        this.moveEnemy('left', enemy);
      }
    }
  });
}
```

El método moveEnemy es el encargado de hacer que el enemigo se mueva en las dos direcciones, además de cambiarles la dirección por si han chocado con alguna plataforma.

En el método update se actualiza al grupo de enemigos si han chocado con alguna plataforma para girarlos a través del método visto anteriormente moveEnemy.

Clase Collectables:

La clase “Collectables” es la encargada de crear los recolectables de cada nivel.

```
export default class Collectables extends Phaser.Physics.Arcade.Group{
  private escena: LevelHandler;

  constructor (escena: LevelHandler, nameObject:string, idObject:string, animObject: string){
    super(escena.physics.world, escena);

    this.escena = escena;

    this.addMultiple(this.escena.mapaNivel.createFromObjects(nameObject,{name:idObject, key: idObject}));
    this.escena.physics.world.enable(this.children.entries);
    escena.anims.create({
      key: animObject,
      frames: idObject,
      frameRate: 20,
      repeat: -1
    });
    this.children.entries.map((collectable:any) => {
      collectable.body.setAllowGravity(false);
      collectable.body.setImmovable(true);
      collectable.play(animObject);
    });
  }
}
```

Es clase según la escena genera los sprites con sus animaciones dependiendo de cada tipo de fruta. Se inmoviliza el objeto, y se ejecuta la animación de forma infinita.

Clase Platforms:

Por último en este punto tenemos los objetos plataformas en la clase “Platforms”, esta clase también extiende de “Group”.

En el constructor se generan las plataformas según el tileset de cada nivel con la particularidad que, dependiendo un parámetro booleano, crea las horizontales o verticales.

```
this.children.entries.map((platform:any) => {
  platform.body.setCollideWorldBounds(true);
  platform.body.setAllowGravity(false);
  platform.body.setImmovable(true);
  platform.body.setSize(32, 8);
  platform.body.setOffset(0, 0);
  platform.play(animObject);
  if(this.horizontal){
    platform.body.setVelocityX(this.velocity);
    platform.body.setFrictionX(1);
    this.movePlatformHorizontal(Phaser.Math.Between(0,1) ? 'left' : 'right', platform);
  } else{
    platform.body.setVelocityY(this.velocity);
    platform.body.setFrictionY(1);
    this.movePlatformVertical(Phaser.Math.Between(0,1) ? 'up' : 'down', platform);
  }
});
```


Aquí estamos recorriendo el mapa de entidades una a una para decirle a cada plataforma que sea inmutable, que no tenga gravedad, que choque con los bordes, el tamaño, offset y velocidad, siendo esta última alternativa entre velocidad vertical o horizontal dependiendo del booleano mencionado anteriormente.

Este método llama a `movePlatformHorizontal` y `movePlatformVertical` para darle el movimiento.

```
movePlatformHorizontal(direction: string, platform: any) {
  if (direction === 'left') {
    platform.body.setVelocityX(-this.velocity);
  }else{
    platform.body.setVelocityX(this.velocity);
  }
}

movePlatformVertical(direction: string, platform: any) {
  if (direction === 'up') {
    platform.body.setVelocityY(-this.velocity);
  }else{
    platform.body.setVelocityY(this.velocity);
  }
}
```

Por último este es el update de la clase:

```
public update(): void{
  this.children.entries.map((platform:any) => {
    if(this.horizontal){
      if(platform.body.velocity.x === 0){
        this.movePlatformHorizontal(Phaser.Math.Between(0,1) ? 'left' : 'right', platform);
      }
      if(platform.body.blocked.right || platform.body.blocked.left) {
        if (platform.body.velocity.x < 0) {
          this.movePlatformHorizontal('right', platform);
        } else {
          this.movePlatformHorizontal('left', platform);
        }
      }
    }else{
      if(platform.body.velocity.y === 0){
        this.movePlatformVertical(Phaser.Math.Between(0,1) ? 'up' : 'down', platform);
      }
      if(platform.body.blocked.up || platform.body.blocked.down) {
        if (platform.body.velocity.y < 0) {
          this.movePlatformVertical('down', platform);
        } else {
          this.movePlatformVertical('up', platform);
        }
      }
    }
  });
}
```

En este método de actualización vemos que recorre el mapa para ir viendo a cada entidad, si es horizontal mira si está chocando con algún terreno en el eje x, si es vertical, mira si se está chocando con algún terreno en el eje y.

Ejecución de desarrollador:

Para poder ejecutar el proyecto en forma de desarrollador deberemos irnos a Visual Studio Code y tener instalado de forma localizada o global en nuestro ordenador la última versión de node.js, esta será nuestra herramienta de arranque del sistema.

Deberemos irnos a una nueva terminal y, localizándonos en la carpeta `template-webpack-ts`, escribir el comando `“npm run dev”`. Con este comando podremos iniciar nuestro proyecto y ver los cambios que programemos al momento.

Compilar y desplegar:

Para desplegar el proyecto debemos compilarlo primero. Compilar el proyecto hace que el framework de Phaser actúe y así transpile el código de TypeScript a JavaScript para que luego se pueda ejecutar en este formato.

Para compilar el proyecto debes ir a una nueva terminal dentro de Visual Studio Code y deberás escribir el comando `“npm run build”`. Con este comando dentro de la carpeta `template-webpack-ts` se habrá creado una carpeta llamada `“dist”` en la que se encontrarán todos los archivos necesarios para poder ejecutar el videojuego.

El proyecto se encuentra desplegado en la página local de `github.io`. Esta página para desarrolladores permite el despliegue de aplicaciones del lado del cliente de forma gratuita siempre que el código esté subido de forma pública al repositorio asociado.

Para el despliegue del proyecto en este repositorio deberemos colocar todos los archivos que están dentro de la carpeta `“dist”` mencionada anteriormente, dentro de la carpeta `“docs”` que se encuentra en la raíz del proyecto. Una vez pegados los archivos en este lugar se deberá hacer un commit y push al repositorio para que los cambios se vean reflejados en github, luego de esto, ya estaría desplegado automáticamente.

SCRIPT DE LA PÁGINA

Alternativamente al juego, el archivo index.html está asociado a un script en formato JavaScript para la visualización de recursos y animaciones enlazados de forma directa al html y su contenido. Por ello, repasaremos el contenido de este script y veremos su funcionalidad al completo.

Este script está diseñado para que se ejecute solamente cuando cargue todo el contenido del html por lo que estableceremos la propiedad DOMContentLoaded.

Modo claro/oscuro:

Empezando vemos la funcionalidad básica de un botón que cuando es presionado cambia los valores para establecer un modo claro y un modo oscuro a la página, esto hace que el usuario tenga mejor experiencia de usuario y personalización.

```
// BOTON MODOS CLARO/OSCURO
const modoToggle = document.getElementById("modo-toggle");

if (localStorage.getItem("modoOscuro") === "true") {
  document.body.classList.add("dark-mode");
  modoToggle.textContent = "☀ Modo claro";
}

modoToggle.addEventListener("click", () => {
  document.body.classList.toggle("dark-mode");

  const esOscuro = document.body.classList.contains("dark-mode");
  modoToggle.textContent = esOscuro ? "🌙 Modo oscuro" : "☀ Modo claro";

  localStorage.setItem("modoOscuro", esOscuro);
});
```

En este código vemos que cuando el botón es clicado se añade o elimina una clase css al body de nuestro html, esta clase es la que desde el css cambiará los colores en función al modo seleccionado.

Además se cambiará el texto del botón y se establecerá un item nuevo en el localStorage del navegador para que sea guardado y visible entre páginas.

Modo pantalla completa:

Siguiendo con la experiencia del usuario hemos añadido un botón de pantalla completa a la página con relación al videojuego con afán de que el usuario pueda jugar cómodamente en el aspecto de pantalla que más le guste.

```
// BOTON FULLSCREEN CONFIG
const fullscreenBtn = document.getElementById("fullscreen-btn");
const gameContainer = document.getElementById("game-container");

fullscreenBtn.addEventListener("click", () => {
  if (gameContainer.requestFullscreen) {
    gameContainer.requestFullscreen();
  } else if (gameContainer.webkitRequestFullscreen) {
    gameContainer.webkitRequestFullscreen();
  } else if (gameContainer.msRequestFullscreen) {
    gameContainer.msRequestFullscreen();
  }
});
```

Animación de paneles:

Para hacer la página más visual y entretenida al usuario se ha decidido que las tarjetas de información de la parte inferior de la página web tengan un pequeño script en el que se identifique cada una de ellas y se establezca una animación de entrada.

```
// ANIMACION DE PANELES
const panels = document.querySelectorAll(".info-panel");

const observer = new IntersectionObserver(
  (entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        entry.target.classList.add("visible");
      }
    });
  },
  {
    threshold: 0.5,
  }
);

panels.forEach((panel) => {
  observer.observe(panel);
});
```

Ranking global:

Una de las características que más les gusta a los usuarios en los videojuegos es la competitividad, por ello, hemos implementado la funcionalidad de un ranking global en el que los jugadores puedan subir su puntuación y participar en una competición sobre quien obtiene más puntos mientras pasa los niveles del videojuego.

```
// DESPLEGABLE RANKING
const toggleBtn = document.getElementById("ranking-toggle");
const content = document.getElementById("ranking-content");
const section = document.getElementById("ranking-section");

content.classList.remove("expanded");
toggleBtn.textContent = "Ranking ▼";

const firebaseConfig = {
  apiKey: "AIzaSyB44_8D563uv-utz8sSSd7E-ap5ukDNP9o",
  authDomain: "frogalone-77.firebaseio.com",
  databaseURL:
    "https://frogalone-77-default-rtdb.europe-west1.firebaseio.com",
  projectId: "frogalone-77",
  storageBucket: "frogalone-77.firebaseio.com",
  messagingSenderId: "1051677992729",
  appId: "1:1051677992729:web:3e14cdeb237608863e4788",
};

const app = initializeApp(firebaseConfig);
const database = getDatabase(app);

const rankingContent = document.getElementById("ranking-content");
```

Este ranking está asociado a unos elementos html con id para hacerles llegar la información, aquí se establece la base de datos firebase con su configuración y si inicia.

```
async function cargarRanking() {
  rankingContent.innerHTML = "<p>Cargando ranking...</p>";

  const dbRef = ref(database);
  try {
    const snapshot = await get(child(dbRef, "jugadores"));
    if (snapshot.exists()) {
      const data = snapshot.val();
      const jugadores = Object.values(data);

      jugadores.sort((a, b) => b.puntuacionTotal - a.puntuacionTotal);

      rankingContent.innerHTML = "";
    }
  } catch (error) {
    console.log(error);
  }
}
```

En esta función que participa en el desarrollo de manera asíncrona cargamos la base de datos a través de los objetos creados en nuestra base de datos de firebase. Además, ordenamos estos valores según su puntuación total para establecer en el html únicamente los 3 mejores jugadores con sus puntuaciones.

```
jugadores.forEach((jugador, index) => {
  if(index <= 3){
    return;
  }
  const card = document.createElement("div");
  card.className = "ranking-card";
  let medallaSrc = "";
  if (index === 0) medallaSrc = "./assets/imagenes/medallas/oro.png";
  else if (index === 1)
    medallaSrc = "./assets/imagenes/medallas/plata.png";
  else if (index === 2)
    medallaSrc = "./assets/imagenes/medallas/bronce.png";

  card.innerHTML = `
    ${
      medallaSrc
      ? ``
      : ""
    }
    <div>
      <h3>${index + 1}. ${jugador.nombre}</h3>
      <p>Puntos: ${jugador.puntuacionTotal}</p>
    </div>
  `;
  rankingContent.appendChild(card);
});
} else {
  rankingContent.innerHTML =
    "<p>No hay datos de ranking disponibles.</p>";
}
} catch (error) {
  console.error("Error al cargar el ranking:", error);
  rankingContent.innerHTML = "<p>Error al cargar el ranking.</p>";
}
}
```

Solamente recogemos los primeros 3, por ello cuando llegamos al tercer índice empezamos a retornar valores vacíos para que no carguen más valores. Añadimos también un asset de medalla de oro, plata o bronce según la posición en la que se encuentre el jugador. Todo esto lo establecemos en el html.

Si no cargara la base de datos o no hubiera registros se recoge a través de un else y un try catch para que el usuario tenga plena consciencia del error y pueda probar a recargar la página o pedir a través del soporte una solución.

```
toggleBtn.addEventListener("click", async () => {
  const isExpanded = content.classList.contains("expanded");

  if (!isExpanded) {
    await cargarRanking();
  }

  content.classList.toggle("expanded", !isExpanded);
  toggleBtn.textContent = isExpanded ? "Ranking ▼" : "Ranking ▲";

  if (!isExpanded) {
    setTimeout(() => {
      section.scrollIntoView({ behavior: "smooth", block: "start" });
    }, 100);
  }
});
```

Este es el botón que se encarga de mostrar y eliminar de la pantalla los datos del ranking, cuando es presionado cargará una animación para establecer el ranking en la mitad de la pantalla.

Comentarios de jugadores:

Como hemos visto antes en la base de datos de firebase tenemos una sección de comentarios absolutamente libre para que los jugadores se expresen y den reseñas sobre el videojuego.

Con la base de datos previamente establecida:

```
const comentarioForm = document.getElementById("comentario-form");
const comentarioTexto = document.getElementById("comentario-texto");
const comentarioNombre = document.getElementById("comentario-nombre");
const comentariosLista = document.getElementById("comentarios-lista");

comentarioForm.addEventListener("submit", async (e) => {
  e.preventDefault();
  const texto = comentarioTexto.value.trim();
  const nombre = comentarioNombre.value.trim();
  if (texto && nombre) {
    const nuevoComentario = {
      nombre,
      texto,
      fecha: new Date().toISOString(),
    };
    await push(ref(database, "comentarios"), nuevoComentario);
    comentarioTexto.value = "";
    comentarioNombre.value = "";
  }
});
```

Recogemos los valores de los inputs del formulario y creamos una constante, luego lanzamos ese comentario directo a la base de datos y reiniciamos los inputs.

Con esto se enviaría el comentario de forma directa a la base de datos junto con la fecha actual.

```
onValue(ref(database, "comentarios"), (snapshot) => {
  comentariosLista.innerHTML = "";
  const data = snapshot.val();
  if (data) {
    const comentarios = Object.values(data).reverse();
    comentarios.forEach((comentario) => {
      const div = document.createElement("div");
      div.className = "comentario";
      div.innerHTML = `${comentario.nombre}</strong> ${comentario.texto}`;
      comentariosLista.appendChild(div);
    });
  }
});
```

Este apartado es el que se encarga de recoger los comentarios subidos y mostrarlos en el apartado del html correspondiente, en principio, luego del formulario de comentarios.

Inteligencia Artificial para chatbot:

A fin de tener una experiencia de usuario plena y un soporte que cumpla los requisitos 24/7 hemos implementado la ayuda de una inteligencia artificial a modo de ChatBot.

Este ChatBot podrás ser visualizado por todos los usuarios si pulsan en la parte inferior derecha en el icono de texto. Aquí podrán escribir cualquier texto que será enviado a la IA y esta lo procesará y responderá en tiempo real.

Para establecer el uso de la inteligencia artificial de Gemini (de uso libre), modelo Gemini IA 2.5 Flash Preview, se ha utilizado un back-end programado en nodejs. Este código puede ser visualizado en el repositorio de github:

<https://github.com/jgargar558j/FrogAloneIAServer>

Repasemos el código:

```
const app = express();
const PORT = 3001;

app.use(cors({
  origin: "https://jgargar558j.github.io",
  methods: ["GET", "POST"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));

app.use(express.json());

const auth = new GoogleAuth({
  credentials: JSON.parse(process.env.GOOGLE_SERVICE_ACCOUNT),
  scopes: "https://www.googleapis.com/auth/generative-language",
});
```

En este apartado establecemos el puerto de escucha y el único origen del que se recibirán peticiones y se enviarán respuestas, esto se hace a modo de seguridad.

Más tarde establecemos a través de una variable de entorno la clave API asociada a la cuenta del desarrollador.

```
app.post("/api/chat", async (req, res) => {
  const { messages } = req.body;

  try {
    const client = await auth.getClient();
    const accessToken = await client.getAccessToken();

    const response = await fetch(
      "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-preview-05-20:generateContent",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${accessToken.token}`,
        },
        body: JSON.stringify({
          contents: [
            {
              role: "user",
              parts: messages.map((msg) => ({ text: msg.content })),
            },
          ],
        }),
      },
    );
  }
});
```

Luego se establece un endpoint al cual haremos la llamada, este endpoint se encarga de a través del token generado como clave API llamará a la url de la inteligencia

artificial y este será el que de la respuesta. Se recogerá procesará y más tarde se enviará al front end para su uso.

```
const data = await response.json();
res.json(data);
} catch (error) {
  console.error("Error al llamar a Gemini:", error);
  res.status(500).json({ error: "Error al procesar la solicitud" });
}
});
```

Este servidor ha sido alojado de forma gratuita en la página web onRender para poder hacer uso de ella, de igual manera aunque sea de uso libre, solamente puede recibir llamadas desde la url de nuestro proyecto.

Visto el servidor para hacer la llamada volvamos al front end.

```
const chatWindow = document.getElementById("chat-window");
const chatToggle = document.getElementById("chat-toggle");

chatToggle.addEventListener("click", () => {
  const isVisible = chatWindow.style.display === "flex";
  chatWindow.style.display = isVisible ? "none" : "flex";
});

const chatInput = document.getElementById("chat-input");
const chatMessages = document.getElementById("chat-messages");

const systemPrompt = `
Eres el desarrollador de FrogAlone, un juego hecho con Phaser y TypeScript. Si alguien
quiere ver detalles técnicos, puede ir al enlace de GitHub que está al final de la página del juego. En
FrogAlone controlas a Héctor, una rana que ha perdido el contacto con otras ranas y ahora salta por los
niveles, pisa enemigos y recoge fruta (le flipa). También le encantan los trofeos brillantes. Se mueve con
A y D o con las flechas izquierda y derecha, y salta con W, flecha arriba o espacio.
Responde siempre en español, de forma clara, muy corta y solo a lo que te pregunten. Usa un tono informal,
como si hablaras con colegas.
Recuerda lo que el usuario te ha dicho antes y responde teniendo en cuenta el contexto de la conversación.
Si el usuario dice "hola", responde con: "¡Qué pasa bala!"
Si el usuario dice "qué pasa bala" o "que pasa bala", responde con: "¡Qué pasa bala!"
`;
```

Establecemos los contenedores del html y desarrollamos un prompt para establecer un sistema ya que queremos que la inteligencia artificial tenga contexto del videojuego y la página web.

```

let conversationHistory = [{ role: "system", content: systemPrompt }];

chatInput.addEventListener("keypress", async function (e) {
  if (e.key === "Enter" && chatInput.value.trim() !== "") {
    const userPrompt = chatInput.value;
    appendMessage("👤", userPrompt);
    chatInput.value = "";

    conversationHistory.push({ role: "user", content: userPrompt });

    // Crear elemento con animación de puntos
    const typingMessage = document.createElement("div");
    typingMessage.innerHTML = `🤖 <span class="typing">escribiendo</span><span class="dots">.</span>`;
    chatMessages.appendChild(typingMessage);
    chatMessages.scrollTop = chatMessages.scrollHeight;

    // Iniciar animación de puntos
    let dotCount = 1;
    const dotsSpan = typingMessage.querySelector(".dots");
    const dotInterval = setInterval(() => {
      dotCount = (dotCount % 3) + 1;
      dotsSpan.textContent = ".".repeat(dotCount);
    }, 400);
  }
});

```

Establecemos que tenga un historial de conversación para que la inteligencia artificial recuerde todo lo que hemos hablado con ella en esta instancia y así preguntarle cosas del pasado. Más tarde cuando el botón de enter es pulsado se recoge del input el texto que ha puesto el usuario y se establece una animación mientras se está haciendo la llamada.

```

try {
  const response = await fetch(
    "https://frogaloneiaserver.onrender.com/api/chat",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        messages: conversationHistory,
      }),
    }
  );

  if (!response.ok) {
    throw new Error("Error en la respuesta de la API");
  }

  const data = await response.json();
  const aiMessage =
    data.candidates?.[0]?.content?.parts?.[0]?.text || "Sin respuesta";

  conversationHistory.push({ role: "assistant", content: aiMessage });

  clearInterval(dotInterval);
  typingMessage.textContent = `🤖 ${aiMessage}`;
} catch (error) {
  console.error("Error:", error);
  clearInterval(dotInterval);
  typingMessage.textContent =
    "🤖 Ocurrió un error al procesar tu mensaje.";
}
}

```

A su vez se hace la llamada a la página web del servidor previamente mencionado establecemos los valores y los enviamos, el servidor es el que procesa y trae una respuesta, recogemos la respuesta y la enviamos al chat sustituyendo el mensaje placeholder de espera que hemos colocado previamente.

Este es el método que se encarga de hacer subir los mensajes al chat, tanto por parte del usuario como por parte de la inteligencia artificial:

```
function appendMessage(sender, text) {  
  const message = document.createElement("div");  
  message.textContent = `${sender} ${text}`;  
  chatMessages.appendChild(message);  
  chatMessages.scrollTop = chatMessages.scrollHeight;  
  return message;  
}
```

PÁGINA ALTERNA:

Asociada a la página principal se ha creado una página alternativa para que el usuario pueda disfrutar de un minijuego creado en JavaScript. Para llegar a ella en una de las tarjetas podemos clicar en el enlace y nos llevará directamente a la página.



Esta es una página sencilla en la que tenemos el modo claro y oscuro previamente mencionado. Tenemos un botón de volver para que el usuario en cualquier momento pueda retroceder a la página principal.

Funcionalidad del juego de memoria; necesitas seleccionar de forma consecutiva 2 cartas del mismo tipo para que se añadan a la puntuación, ganas si seleccionas correctamente todas las parejas de cartas.

Para ello:

```
function shuffleArray(array) {
  for (let i = array.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [array[i], array[j]] = [array[j], array[i]];
  }
}

shuffleArray(cardArray)
```

Ordenaremos de forma aleatoria las cartas.

Creamos el tablero y métodos necesarios:

```
function createBoard() {
  for (let i = 0; i < cardArray.length; i++) {
    const cardContainer = document.createElement('div');
    cardContainer.classList.add('card');
    cardContainer.setAttribute('data-id', i);

    const front = document.createElement('img');
    front.classList.add('front');
    front.setAttribute('src', cardArray[i].img);

    const back = document.createElement('img');
    back.classList.add('back');
    back.setAttribute('src', 'images/x3/card_back.png');

    cardContainer.appendChild(front);
    cardContainer.appendChild(back);
    cardContainer.addEventListener('click', flipcard);

    grid.appendChild(cardContainer);
  }
}

function disableAllCards() {
  const cards = document.querySelectorAll('.card');
  cards.forEach(card => card.removeEventListener('click', flipcard));
}

function enableAllCards() {
  const cards = document.querySelectorAll('.card');
  cards.forEach(card => {
    const cardId = card.getAttribute('data-id');
    if (!cardsWon.includes(cardArray[cardId].name)) {
      card.addEventListener('click', flipcard);
    }
  });
}
```

```

function checkforMatch() {
  const cards = document.querySelectorAll('.card');
  const optionOneId = cardsChosenId[0];
  const optionTwoId = cardsChosenId[1];

  const cardOne = cards[optionOneId];
  const cardTwo = cards[optionTwoId];

  if (cardsChosen[0] === cardsChosen[1]) {
    cardOne.removeEventListener('click', flipcard);
    cardTwo.removeEventListener('click', flipcard);
    cardsWon.push(cardsChosen[0]);
  } else {
    setTimeout(() => {
      cardOne.classList.remove('flipped');
      cardTwo.classList.remove('flipped');
    }, 500);
  }

  cardsChosen = [];
  cardsChosenId = [];

  enableAllCards();
  resultDisplay.textContent = cardsWon.length;

  if (cardsWon.length === cardArray.length / 2) {
    resultDisplay.textContent = '¡HAS GANADO!';
  }
}

function flipcard() {
  const cardId = this.getAttribute('data-id');
  if (cardsChosenId.includes(cardId)) return;

  this.classList.add('flipped');
  cardsChosen.push(cardArray[cardId].name);
  cardsChosenId.push(cardId);

  if (cardsChosen.length === 2) {
    disableAllCards();
    setTimeout(checkforMatch, 500);
  }
}

createBoard();

```

Con estos métodos establecemos la funcionalidad del juego simple, cuando seleccionas 2 cartas, se bloquea las elecciones mientras se verifica que las dos cartas sean o no idénticas. Si lo son se quedan boca arriba, si no lo son vuelven a estar boca abajo, las vueltas a las cartas se hacen a través de unas propiedades de animación css.

Manual del usuario

Bienvenido al manual de usuario de FrogAlone, en este documento encontrarás las funciones que tiene el videojuego. Estas funciones son muy básicas, pero si estás leyendo esto es porque puede que te hayas atascado en algún punto, ¡VAMOS A ELLO!

COMO INICIAR EL JUEGO

FrogAlone es un videojuego creado para todos los dispositivos sin necesidad de ser descargado, por eso solo debes ir a la página web:

<https://jgargar558j.github.io/PI2025/>

Solo haciendo esto ya estarás dentro del juego y podrás jugar todo lo que tu quieras.

MENÚS E INTERFAZ

Vamos a ver los menús para que te quede claro para lo que sirve cada uno (aunque tengas un título gigante en el que te lo pone).

Menú principal:

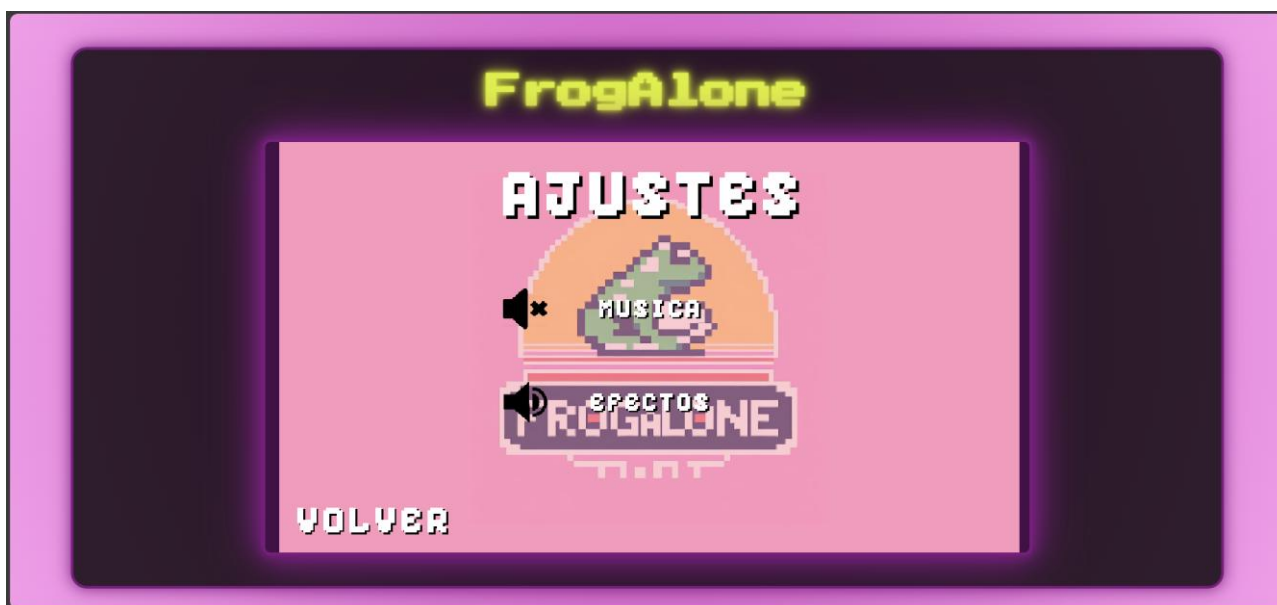
Este es el menú principal;



Aquí podrás clicar en cualquiera de los 3 botones para acceder a sus apartados. Te recomendamos ir al apartado de créditos para que veas quién es el señor tan guapo que ha creado el videojuego 😊.

Ajustes:

En la ventana ajustes podrás activar o desactivar el volumen de la música y de los efectos únicamente clicando en los iconos de volumen. Pruébalo si te molesta la música que con tanto mimo hemos escogido 😊.



Para volver al menú principal simplemente clicando a volver puedes hacerlo.

Créditos:

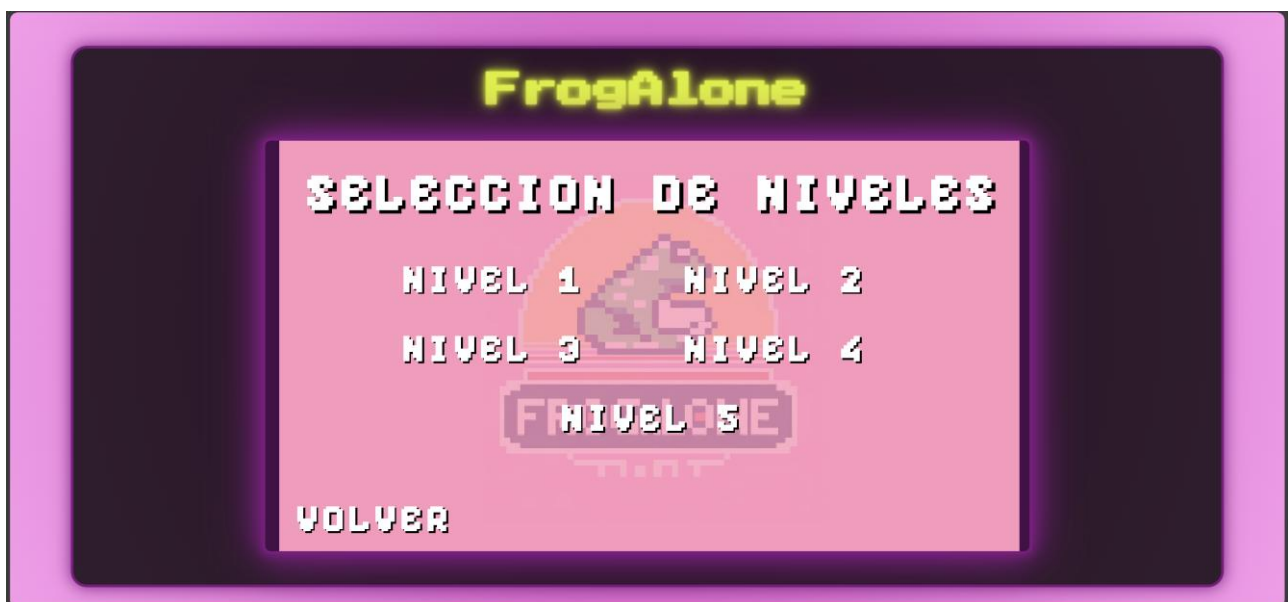
Claramente vamos a repasar la pestaña de créditos por que nos parece que DEBES conocer quien ha creado esta historia de la ranita, para eso se lo ha currado.



Para volver al menú principal simplemente clicando a volver puedes hacerlo.

Selección de niveles:

En la selección de niveles adivina que puedes hacer, si pequeñín, ¡seleccionar nivel!
¿Quién lo diría? Aprovecha y clicas en uno de los niveles para poder jugarlo, te recomendamos que si te has preguntado que se hace en la selección de nivel juegues el tutorial, no tenemos tantas esperanzas en ti...



Para volver al menú principal simplemente clicando a volver puedes hacerlo.

Niveles:

Para completar el nivel debes tocar la copa que se encuentra normalmente al final del mapa. Para ello debes sortear distintos obstáculos como huecos al vacío, plataformas móviles y enemigos.



Completar el nivel tiene un tiempo límite, si te pasas de este tiempo, despídete de las oportunidades para poder conseguir la copa porque, aunque tengas todas las vidas, habrás perdido por falta de tiempo. Como diría el desarrollador del juego, espabila.

Antes hemos hablado de las vidas, pero ¿cómo se pueden ganar y perder vidas? Lo primero de todo es que las vidas restantes pueden ser vistas en la parte superior izquierda.



Facilísimo, perder vidas puede ocurrir cuando un enemigo te golpea o cuando caes al vacío por un hueco. Si te ocurre esto último además de hacerte daño serás teletransportado al inicio del mapa para que vuelvas a empezar y pensar en lo que has hecho.

¿Y ganar vidas? No se puede. Espabila.

Si has perdido alguna vida tendrás que replantearte lo que has hecho y pensarte dos veces las cosas para la próxima vez que tengas intención en perder alguna vida.

La puntuación puede ser vista justo debajo de las vidas, podrás sumar puntuación si recoges frutas, matas enemigos o terminas rápido el nivel. Aprovecha y recoge los máximos puntos posibles.



El tiempo restante puede ser visto en la mitad superior de la pantalla, como hemos mencionado antes, no te quedes mirando las musarañas y completa el nivel antes de que termine el tiempo, si no lo haces, perderás el nivel y 4 minutos de tu vida.

Si completas el nivel y aún quedan suficientes segundos como para que se sepa que no eres un niño de 3 años haremos un cálculo para sumarlo a la puntuación. Esto significa que sí, hay speedruns de puntuación.

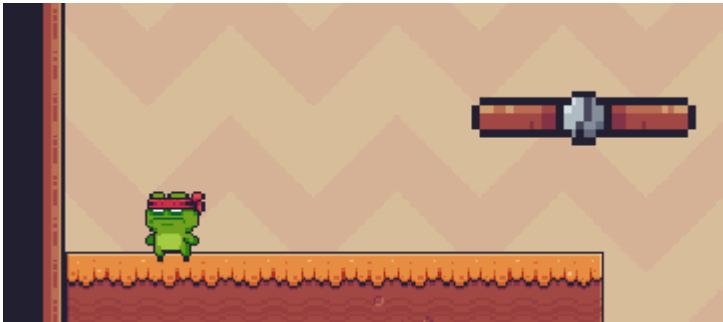


Obstáculos y enemigos:

Lo único que puede impedir tu paso hacia la victoria son los obstáculos y los enemigos. Como obstáculos tenemos los huecos al vacío y las plataformas móviles, uno te mata y te manda al inicio y el otro solo te molesta en la forma de jugar.



En estos huecos podrás acabar con tu vida si quieres, pero no te lo recomendamos.



Estas plataformas pueden ir de manera horizontal o vertical, en cada uno de los casos te llevarán a un lugar muy bonito, o muy feo. Piénsatelo dos veces antes de montarte en una.

Los enemigos que existen en el juego irán de un lado hacia el otro corriendo hasta que choquen con algún terreno, juega esto a tu favor para poder matarlos fácilmente pisándoles la cabeza. Esto hará que exploten y te den puntuación.



Propuesta de ampliación del proyecto

FrogAlone constituye una base sólida sobre la cual se pueden implementar múltiples mejoras y funcionalidades diferentes y novedosas. A continuación, se presentan diversas propuestas de ampliación que permitirán enriquecer la experiencia del usuario, aumentar la escalabilidad del proyecto y explorar nuevas oportunidades técnicas y comerciales.

SISTEMA DE USUARIOS Y AUTENTICACIÓN

Implementar un sistema de registro e inicio de sesión permitiría a los jugadores guardar su progreso en la nube, competir en rankigs globales y personalizar su perfil.

RANKING ONLINE Y LOGROS

Agregar un sistema de puntuaciones globales y logros debloqueables fomentaría la competitividad y la rejugabilidad. Esto incluiría:

- Tabla de clasificación online.
- Sistema de logros con notificaciones en el juego.
- Integración con redes sociales para compartir resultados.

MEJORAS GRÁFICAS Y SONORAS

Integración de nuevos sprites, y nuevas bandas sonoras, así como de nuevos efectos de sonido.

MÁS NIVELES Y ENEMIGOS

La agregación de nuevos niveles es el siguiente paso para seguir en cuanto al desarrollo de este proyecto, además queremos generar una historia paralela a la evolución de los niveles para que el usuario se integre más en el mundo. Agregar a su vez nuevos enemigos que impliquen un reto diferente a los existentes, con nuevas funcionalidades como hacer daño con rango alto, enemigos que quiten más vida... Y así implementar un sistema de mejora de personaje, armadura, más vida...

Estudio de viabilidad del proyecto

DESCRIPCIÓN DE LA IDEA DE NEGOCIO

El proyecto consiste en el desarrollo y comercialización de un videojuego de plataformas 2D accesible desde navegadores web y dispositivos móviles. El juego combina mecánicas clásicas con la actualización de los mecanismos del presente y una jugabilidad fluida, orientado a todo tipo de público: desde jugadores casuales hasta personas con gran recorrido a través del género retro de plataformas.

ANÁLISIS DE MERCADO

El mercado de los videojuegos indie y de navegador ha experimentado un crecimiento constante, impulsado por la accesibilidad, la nostalgia por los juegos retro y la facilidad de distribución digital. Plataformas como Itch.io, Steam y Google Play ofrecen canales viables para la publicación y monetización del juego.

Además, el uso de tecnologías web como Phaser y TypeScript permite un desarrollo ágil, multiplataforma y con bajos costes de infraestructura.

PROPUESTA DE FORMA JURÍDICA

Para formalizar el proyecto como negocio, se propone la constitución de una Sociedad Limita (S.L.) por las siguientes razones:

- Responsabilidad limitada de los socios.
- Facilidad de constitución y gestión.
- Posibilidad de contratar empleados y emitir facturas.
- Acceso a ayudas públicas y subvenciones para emprendedores tecnológicos.

PLAN DE RECURSOS HUMANOS

En una primera fase, el equipo mínimo necesario para el desarrollo y lanzamiento del videojuego estaría compuesto por:

- Desarrollador principal = Programación del juego, lógica, físicas, integración web.
- Diseñador gráfico = Diseño de personajes, escenarios, interfaz.

- Compositor de sonido = Música de fondo, efectos sonoros.
- Tester = Pruebas de jugabilidad y detección de errores.
- Marketing y comunicación = Difusión en redes, contacto con prensa y gestión de comunidad.

En fases posteriores, se podría ampliar el equipo con perfiles especializados en backend, monetización y soporte técnico.

Conclusión

El desarrollo de FrogAlone ha permitido consolidar mis conocimientos técnicos en programación web, diseño de videojuegos y gestión de proyectos. A lo largo del proceso, se ha demostrado la viabilidad de crear una aplicación interactiva, funcional y escalable utilizando tecnologías open source y accesibles.

Entre los logros más destacados se encuentran:

- La implementación de una arquitectura modular y mantenible.
- La creación de una experiencia de juego fluida y atractiva.
- La documentación detallada de cada fase del desarrollo, desde el análisis de requisitos hasta la propuesta de ampliación

Además, el proyecto ha servido como una introducción práctica al uso de estructuras de datos, almacenamiento local, diseño de interfaces y planificación de recursos. También ha puesto de manifiesto el potencial de este tipo de desarrollos para convertirse en un producto comercial, sobre todo, si se amplían las funcionalidades previamente mencionadas.

En resumen, este proyecto no solo ha cumplido los objetivos iniciales, sino que ha abierto nuevas posibilidades de crecimiento y profesionalización en el ámbito del desarrollo de videojuegos web.